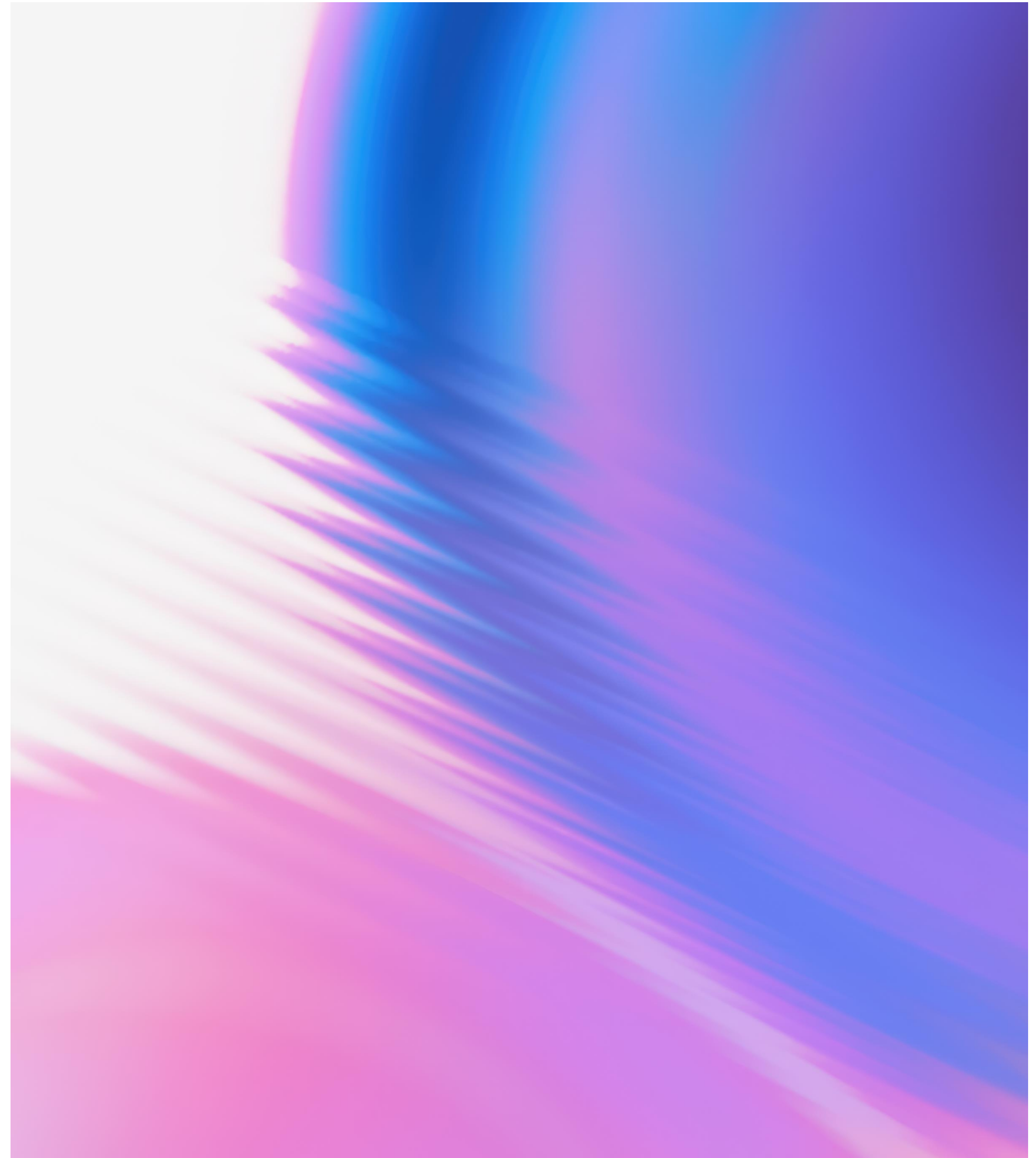
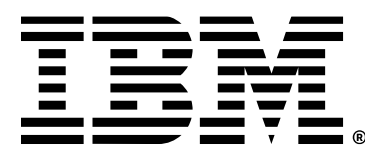


Qiskit Architecture

Matthew Treinish

Qiskit Architect – Senior Technical Staff
Member (STSM)

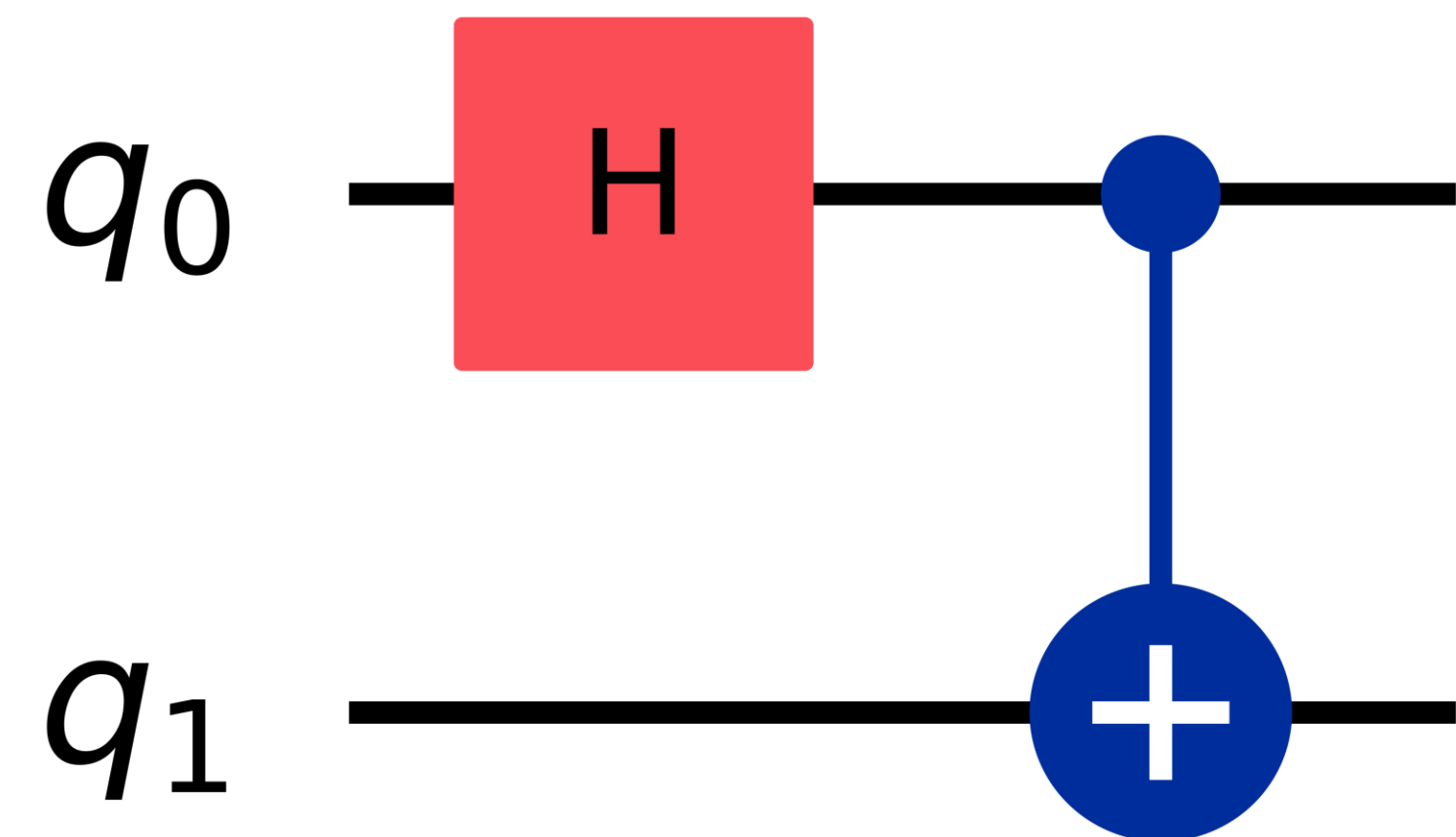
matthew.treinish@ibm.com



What is Qiskit?

- Open source SDK for Quantum Computing
- Apache 2.0 licensed
- Written in Rust and provides a public Python and C API
- Collaboratively developed on [Github](https://github.com/Qiskit/qiskit) with 649 contributors to-date
- Project started in 2017 after launch of IBM QuantumExperience originally to provide Python interface to platform
- Designed to provide hardware agnostic interface for working with Quantum computers.
- Includes a modular quantum circuit compiler to compile abstract circuits to run on quantum computers.

```
1 from qiskit import QuantumCircuit
2 from qiskit.quantum_info import SparsePauliOp
3 from qiskit.transpiler import generate_preset_pass_manager
4 from qiskit_ibm_runtime import EstimatorV2 as Estimator
5
6 # Create a new circuit with two qubits
7 qc = QuantumCircuit(2)
8
9 # Add a Hadamard gate to qubit 0
10 qc.h(0)
11
12 # Perform a controlled-X gate on qubit 1, controlled by qubit 0
13 qc.cx(0, 1)
14
15 # Return a drawing of the circuit using Matplotlib ("mpl"). This is the
16 # last line of the cell, so the drawing appears in the cell output.
17 # Remove the "mpl" argument to get a text drawing.
18 qc.draw("mpl")
```



Qiskit Design Philosophy

Extensible

Designed to be extended and facilitate sharing and reusing research. Provides plug-in interfaces to integrate extensions to the capabilities in the library.

Performance and ease of use

Mixes a performance-oriented core written in the Rust programming language with a flexible and easy to use Python interface and a lower-level C interface.

Portability across hardware

Uses universal circuit representations and transformations that are agnostic to the underlying hardware.

Interoperability across abstraction levels

Qiskit provides several abstraction levels depending on the use case and provides tooling to move between abstraction levels

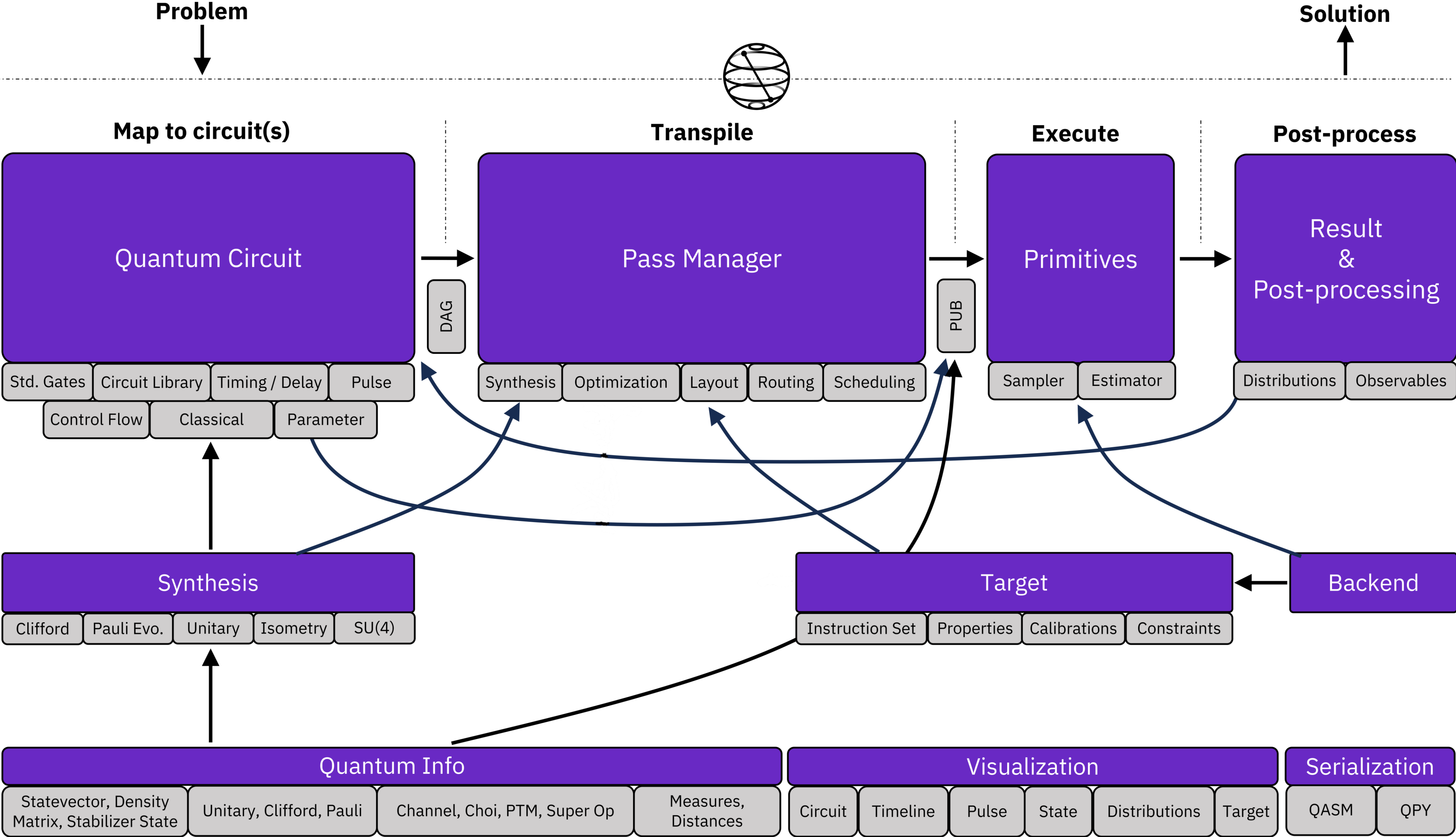
Computational Primitives

There are two main primitives for capturing the output of a quantum circuit: sampling output bitstrings or estimating observable expectation values. These primitives are how circuits are evaluated in Qiskit and using a common abstract interface the implementation details can vary significantly.

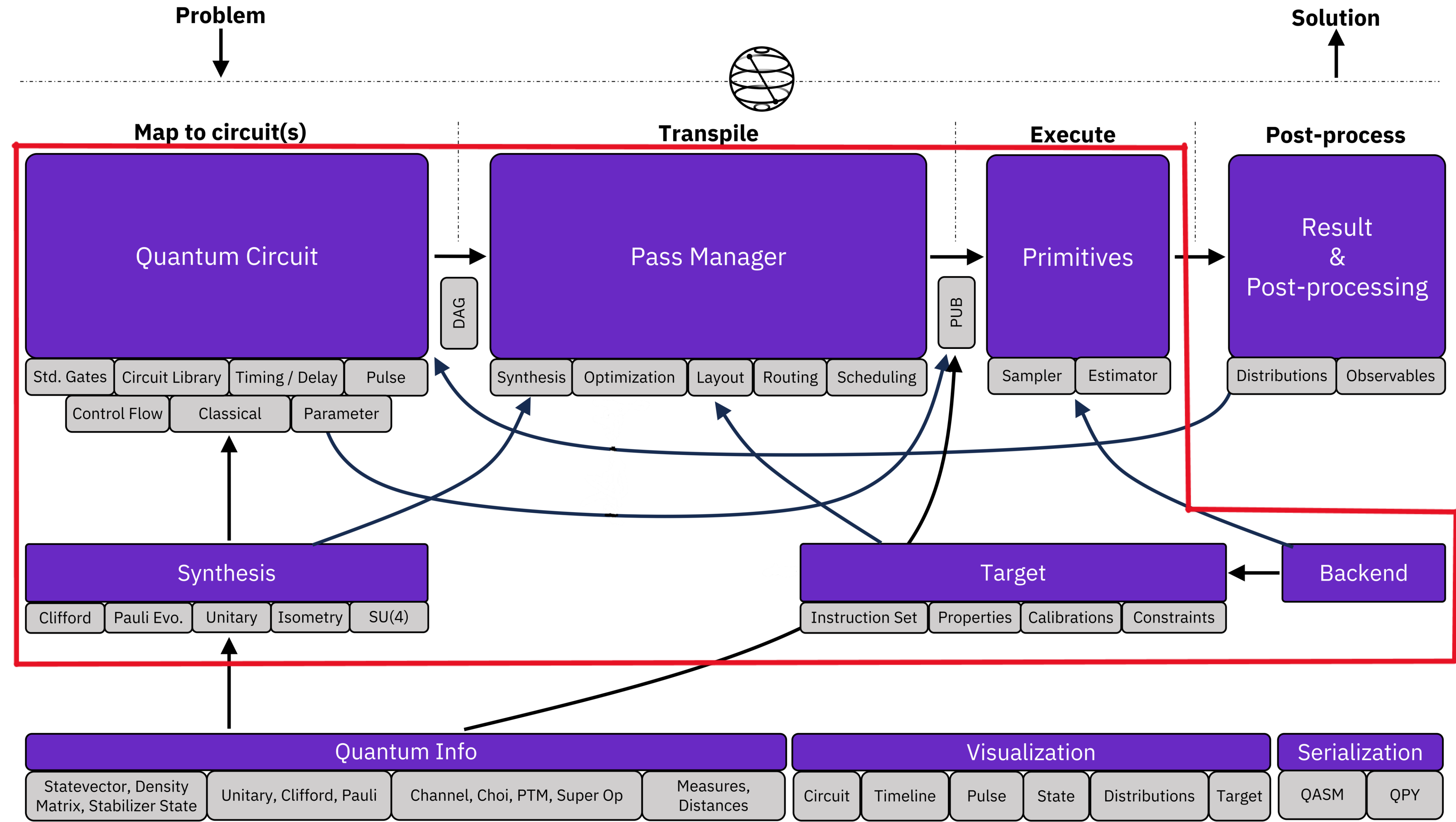
Integrated visualizations

Qiskit includes visualization tools to display large circuits, target backend properties, and state and distributions. The goal is to provide visualizations to aid in debugging a workflow.

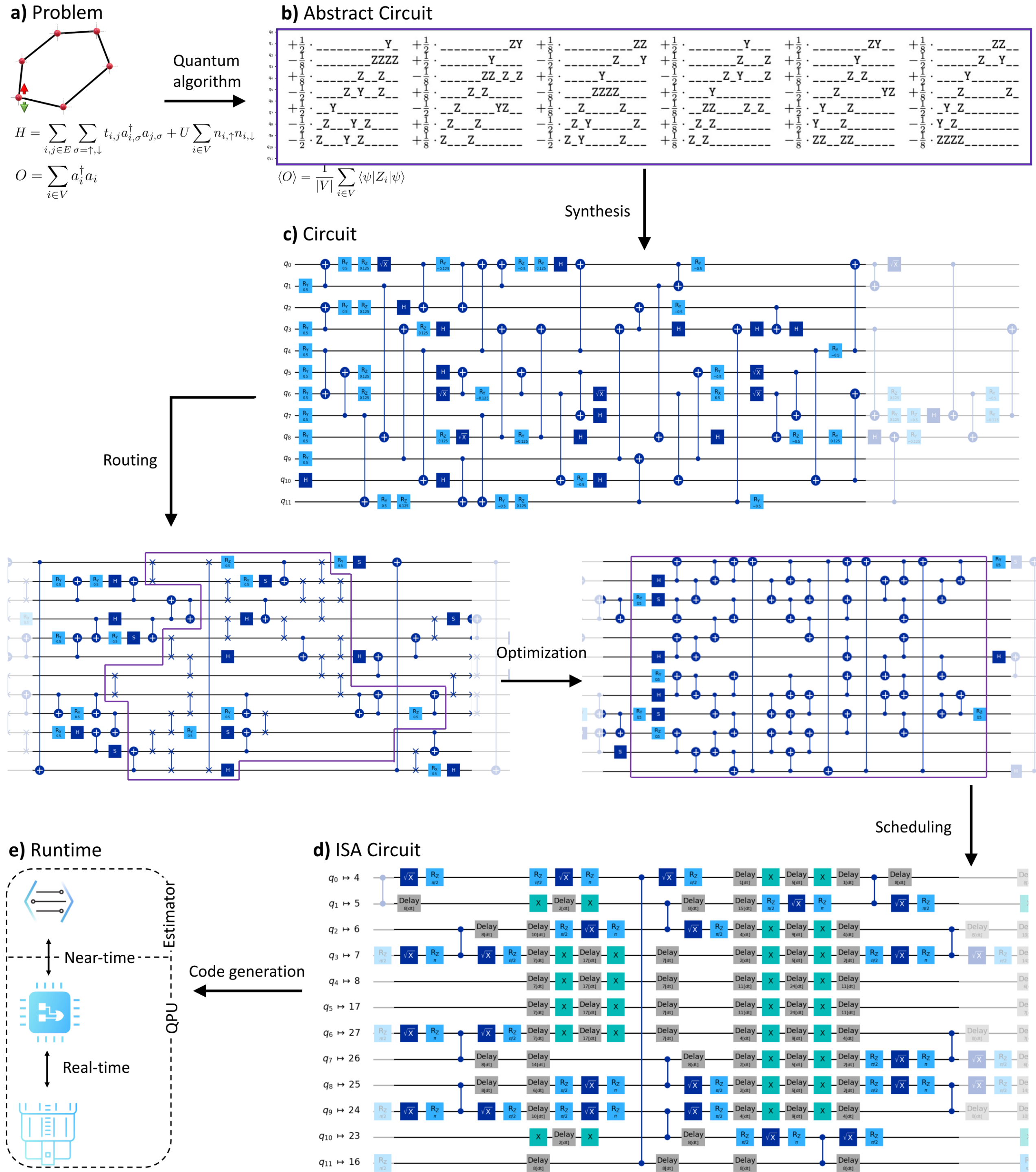
Qiskit Architecture



Qiskit Architecture



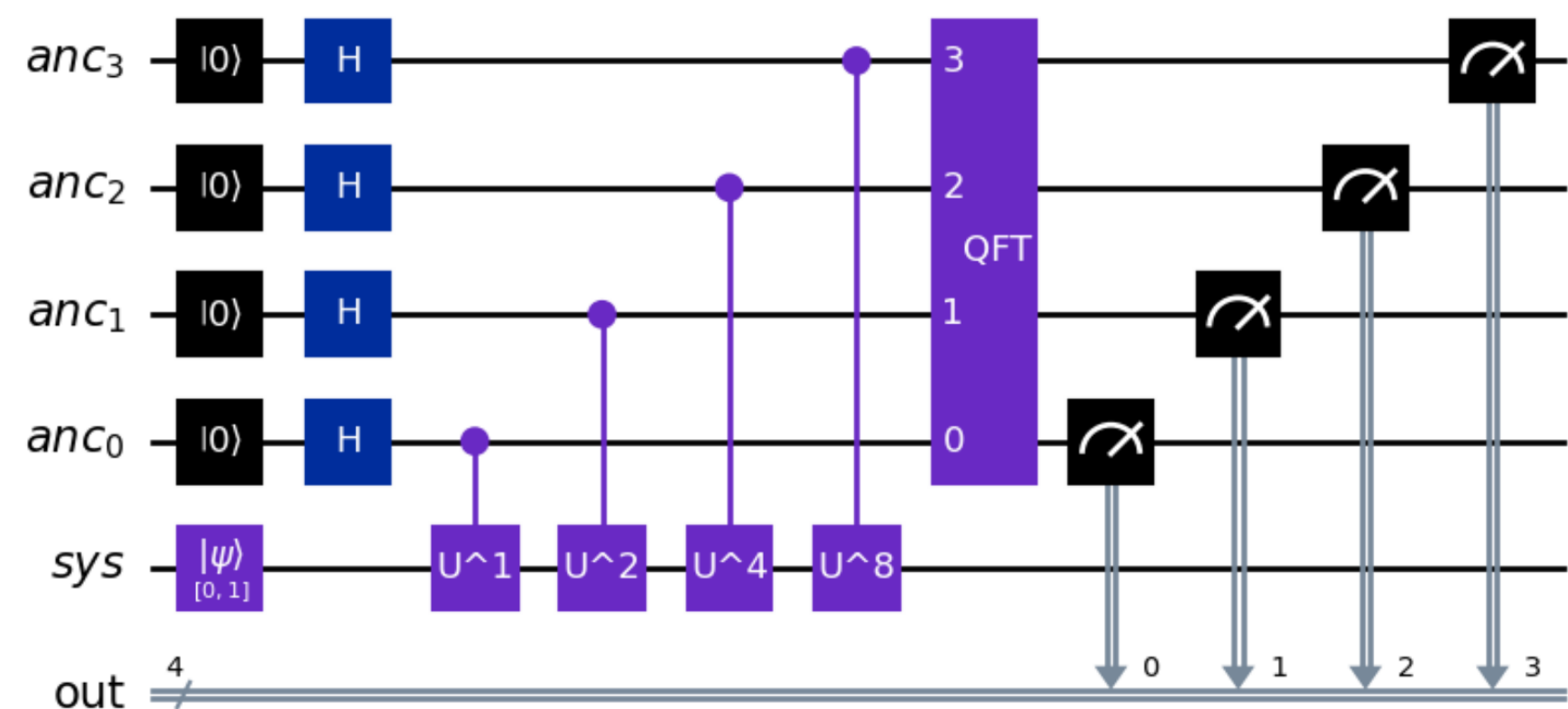
Typical workflow



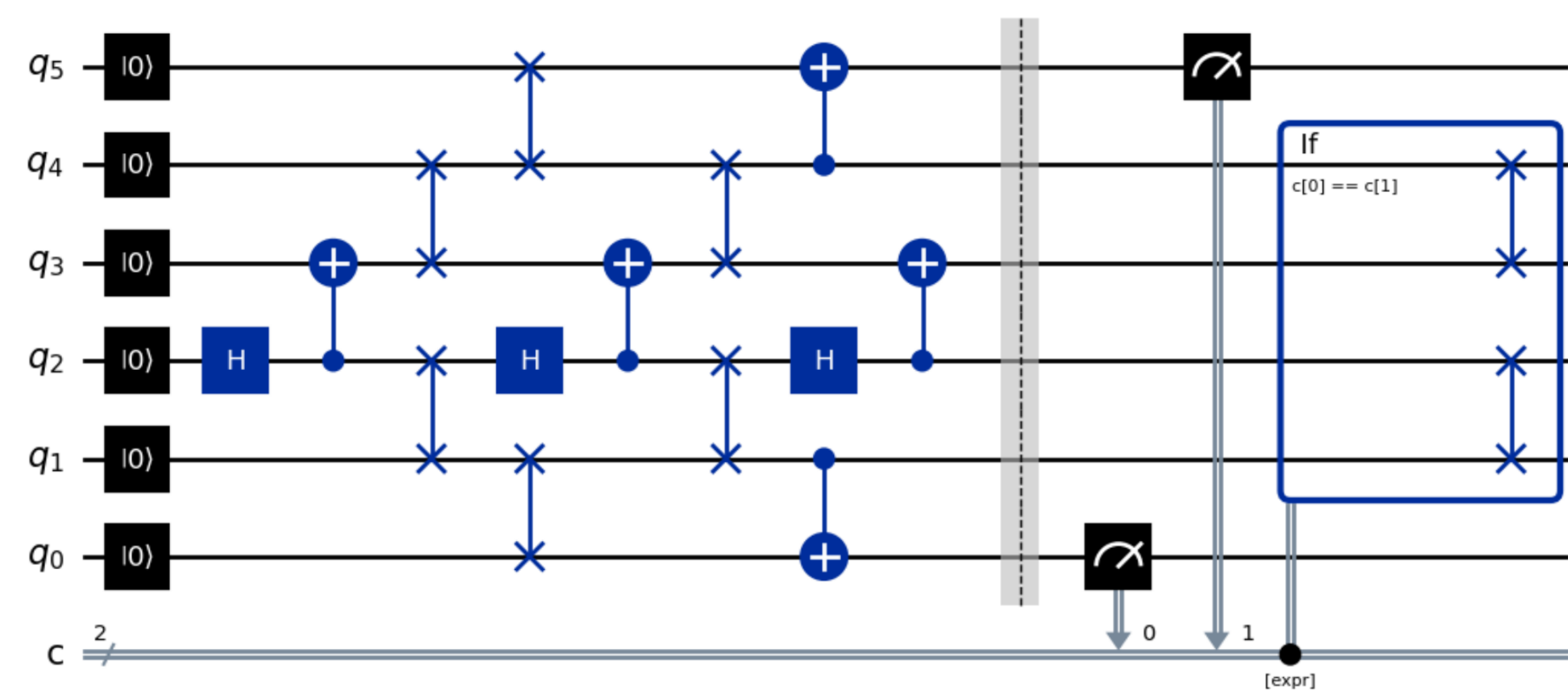
Qiskit Circuits

Qiskit Circuit Abstraction Levels

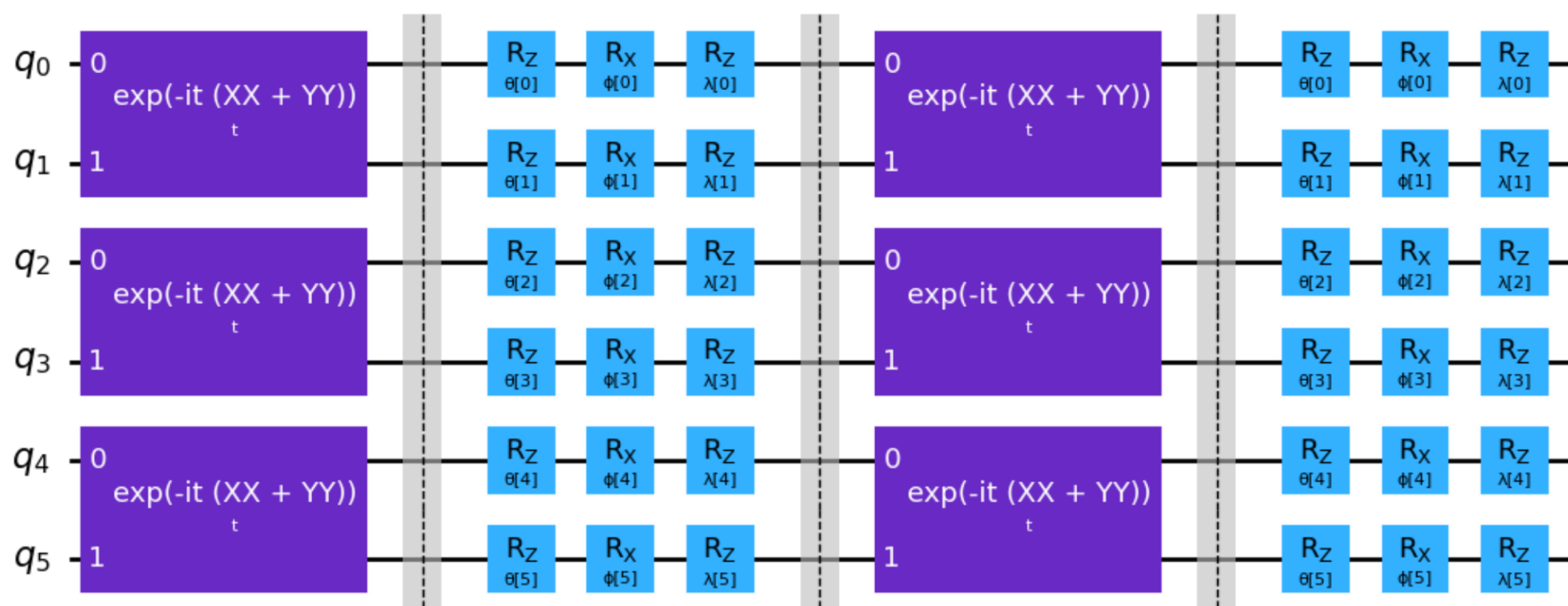
a) High-level abstract circuit composition
(example: quantum phase estimation)



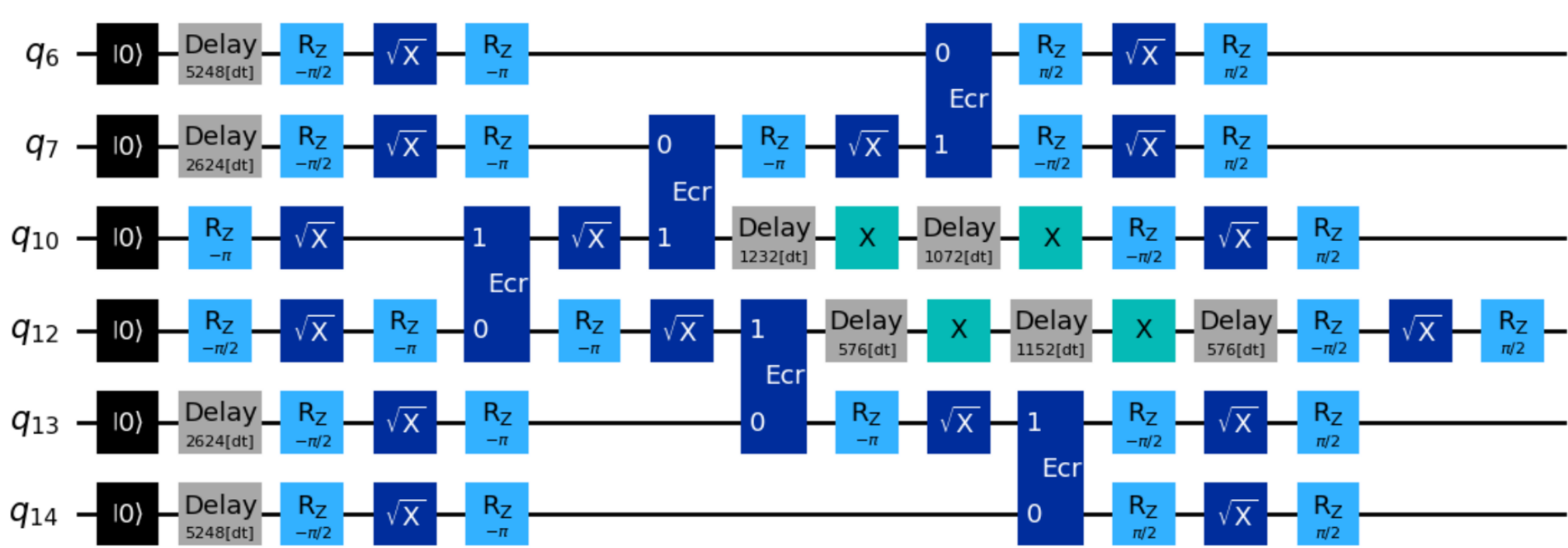
b) Standard gates and classical control flow
(example: entanglement distillation)



c) Parameterized circuits
(example: twirled Trotterized evolution)



d) Hardware native gates and timing (ISA circuit)
(example: GHZ with dynamical decoupling)

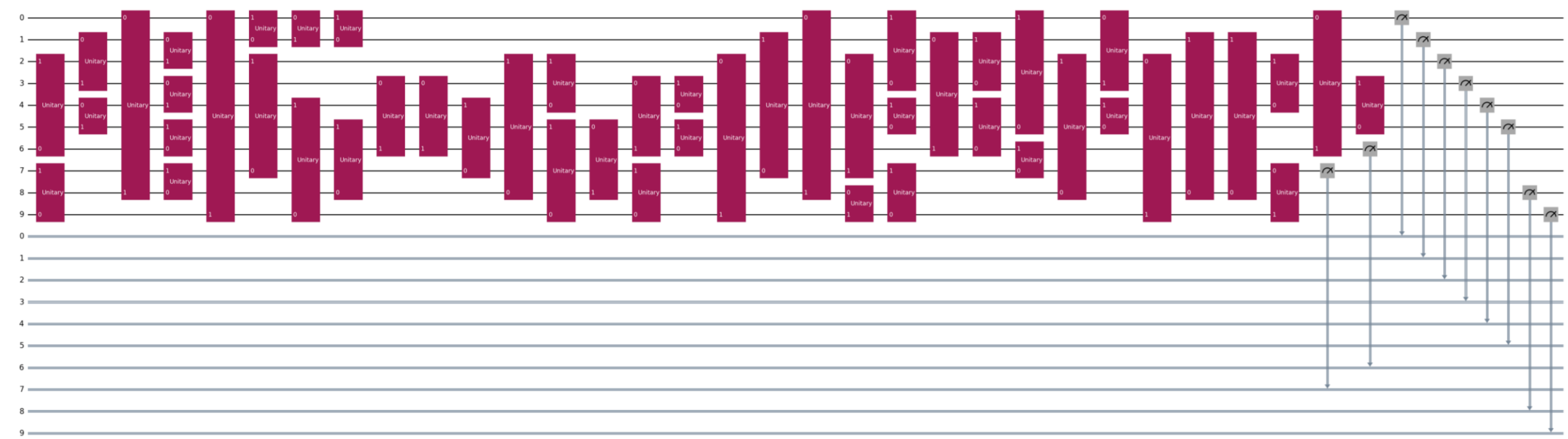


Circuit Library

- Qiskit provides a circuit library that defines numerous operation types and circuits to use as building blocks for applications, serve as benchmarks, or they are circuits conjectured to be difficult to simulate classically..
- This includes everything from standard textbook gates like CX, Toffoli, and Hadamard
- There are also abstract gates like Clifford and Linear Function which are represented by the abstract mathematical operation.
- The library includes generators for creating circuit objects.
- https://quantum.cloud.ibm.com/docs/en/api/qiskit/circuit_library

```
from qiskit.circuit.library import quantum_volume
from qiskit.circuit import Clbit

qc = quantum_volume(10, seed=42)
qc.add_bits([Clbit() for _ in range(10)])
qc.measure(qc.qubits, qc.clbits)
```



Defining Custom Operations

- Qiskit provides mechanisms to define custom operations.
- There are 2 base types for operations, either `Instruction` or `Gate` (a unitary instruction)
- The simplest option is to embed a circuit into a one-off `Instruction` or `Gate` object with `QuantumCircuit.to_instruction()` or `QuantumCircuit.to_gate()`.
- You can also define custom subclasses. These custom operations can contain any representation, the base class's interface describes how Qiskit can interact with.
- In Python there is also support for defining singleton instructions to improve performance for unparameterized and stateless operation types.
- <https://quantum.cloud.ibm.com/docs/en/api/qiskit/circuit#creating-custom-instructions>

```
from math import pi, sqrt
import numpy as np

from qiskit.circuit.singleton import SingletonGate
from qiskit.circuit import QuantumCircuit

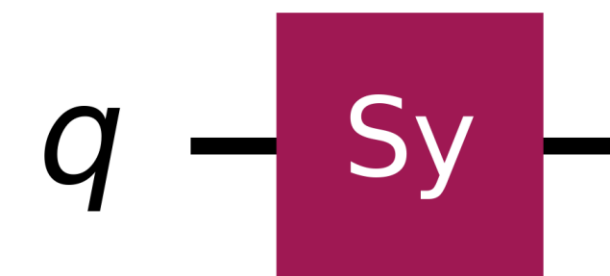
FRAC_1_SQRT_2 = 0.707106781186547524400844362104849039
RY_ARRAY = [[FRAC_1_SQRT_2, -FRAC_1_SQRT_2], [FRAC_1_SQRT_2, FRAC_1_SQRT_2]]

class SYGate(SingletonGate):
    def __init__(self, label=None):
        super().__init__("sy", 1, [], label=label)

    def _define(self):
        qc = QuantumCircuit(1)
        qc.ry(pi / 2, 0)
        self.definition = qc

    def __array__(self, dtype=None, copy=None):
        if dtype is None:
            dtype = np.complex128
        return np.array(RY_ARRAY, dtype=dtype, copy=copy)

qc = QuantumCircuit(1)
qc.append(SYGate(), [0])
```



Defining Abstract Custom Operation

```
import numpy as np

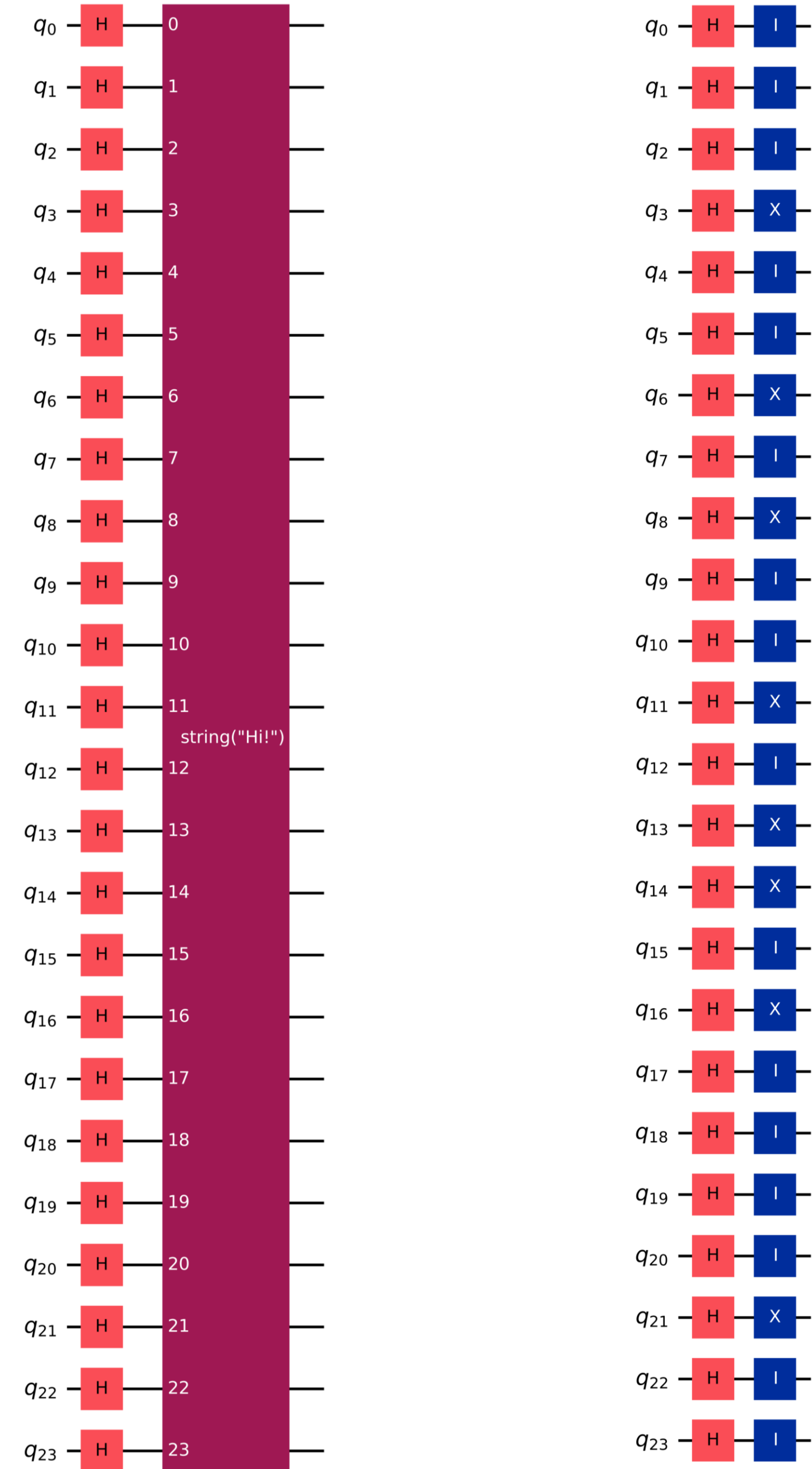
from qiskit.circuit import Gate, QuantumCircuit

class QuantumString(Gate):
    """An entangled string object."""

    def __init__(self, string: str, label=None):
        if not string.isascii():
            raise ValueError("Only ascii strings are valid")
        num_qubits = 8 * len(string)
        self.string = string
        self.string_array = np.frombuffer(string.encode("ascii"), dtype=np.uint8)
        label = f'string("{self.string}")'
        super().__init__("string", num_qubits, [], label=label)

    def _define(self):
        qc = QuantumCircuit(self.num_qubits)
        for i, val in enumerate(self.string_array):
            for j in range(8):
                if val & 1:
                    qc.x(i * 8 + j)
                else:
                    qc.id(i * 8 + j)
                val = val >> 1
        self.definition = qc

string_gate = QuantumString("Hi!")
qc = QuantumCircuit(string_gate.num_qubits)
qc.h(qc.qubits)
qc.append(string_gate, qc.qubits)
```

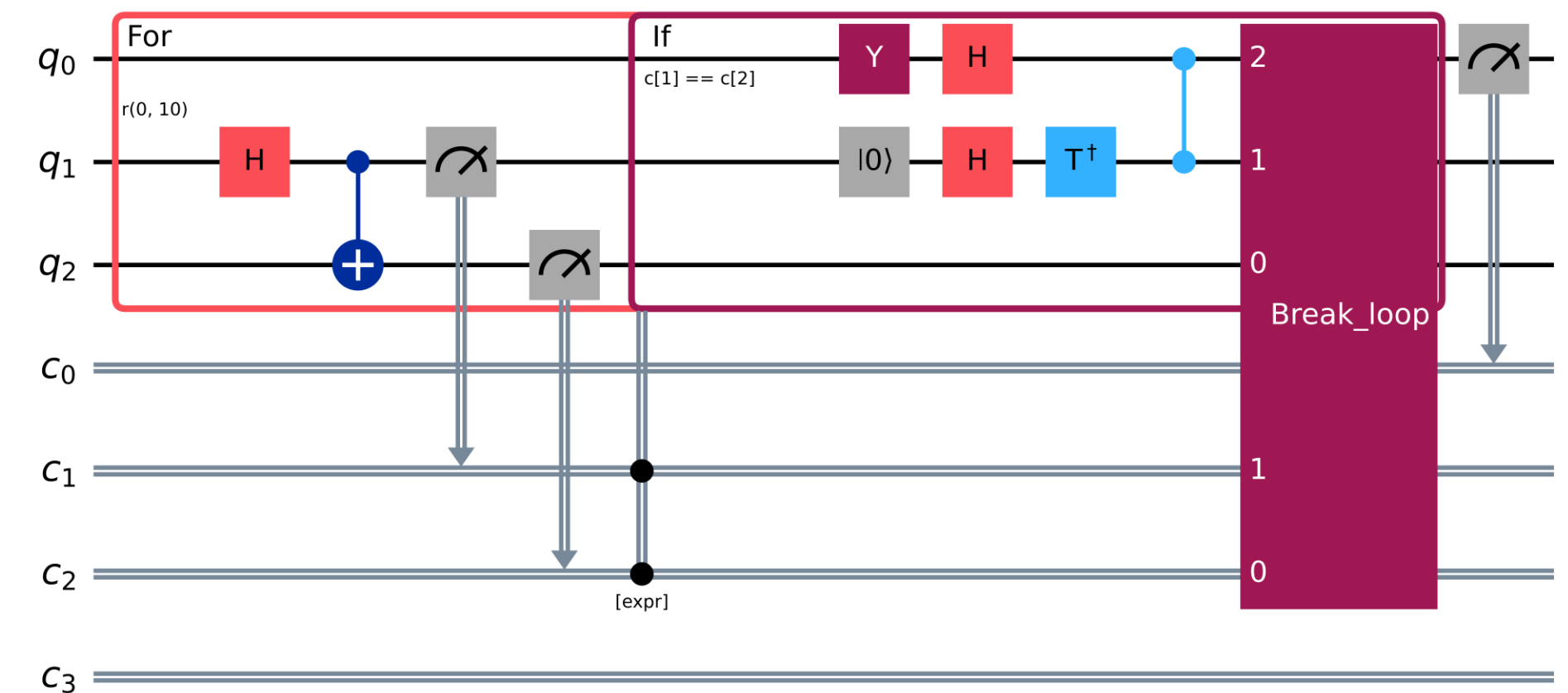


Control Flow and Expression System

- Qiskit enables defining circuits with real-time classical operations that are designed to be executed within the coherence time of the circuit.
- Given these constraints the expressiveness of the classical operations are limited
- Not all hardware supports these capabilities

- <https://quantum.cloud.ibm.com/docs/en/api/qiskit/circuit#real-time-classical-computation>
- https://quantum.cloud.ibm.com/docs/en/api/qiskit/circuit_classical#module-qiskit.circuit.classical

```
import qiskit
from qiskit.circuit.classical import expr
qc = qiskit.QuantumCircuit(3, 4)
with qc.for_loop(range(10)):
    qc.h(1)
    qc.cx(1, 2)
    qc.measure([1, 2], [1, 2])
    with qc.if_test(expr.equal(qc.clbits[1], qc.clbits[2])):
        qc.reset(1)
        qc.h(1)
        qc.tdg(1)
        qc.y(0)
        qc.h(0)
        qc.cz(0, 1)
        qc.break_loop()
qc.measure(0, 0)
```



Qiskit Backend Interface

Overview of the Qiskit Backend model

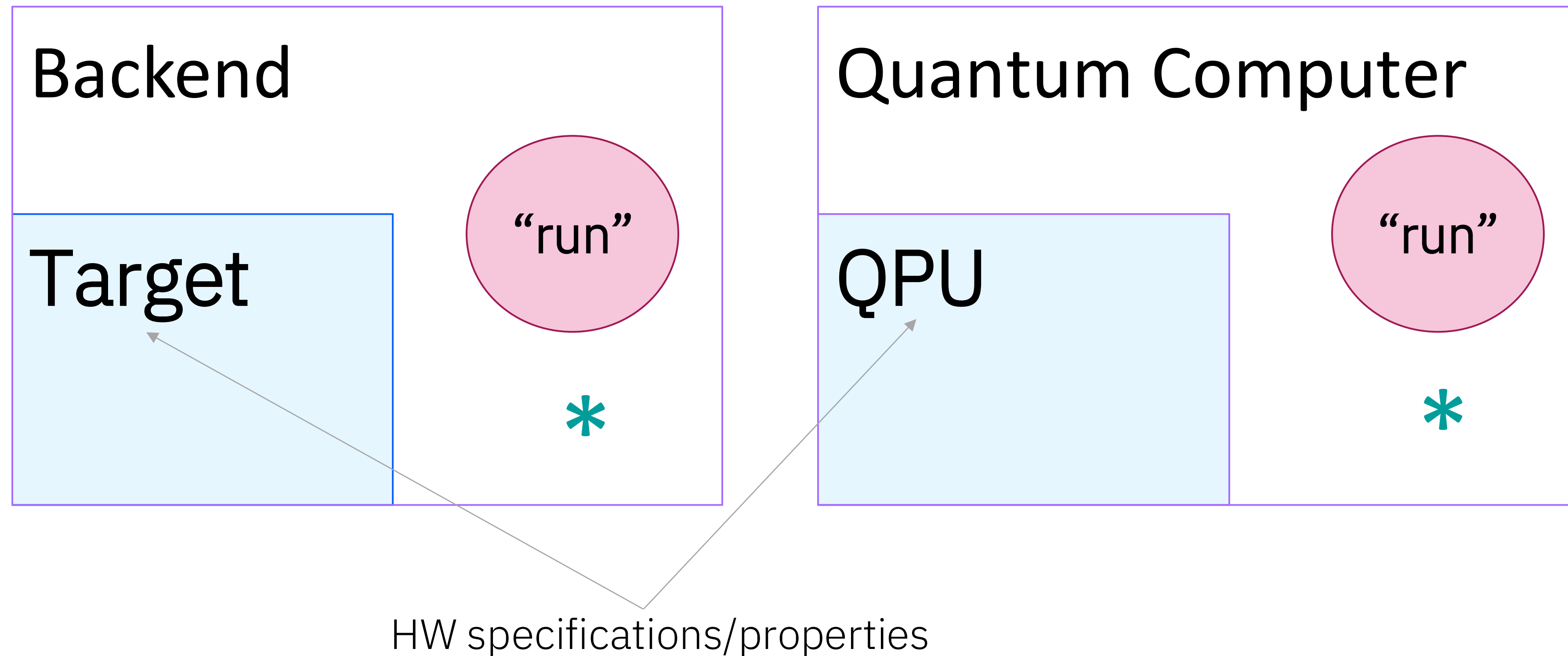
Current definitions:

- **QPU:** *“A QPU includes all of the hardware responsible for accepting an executable quantum instruction set, or a quantum circuit, and returning an accurate answer”*
Qiskit representation → **Target**
- **Quantum Computer:** QPU + infrastructure to run tasks on it
Qiskit representation → **Backend (& Primitives)**

The Backend model

For Qiskit →

Backend = Target + .run + custom transpiler interface
(+ extra functionality)



BackendV2 abstract interface

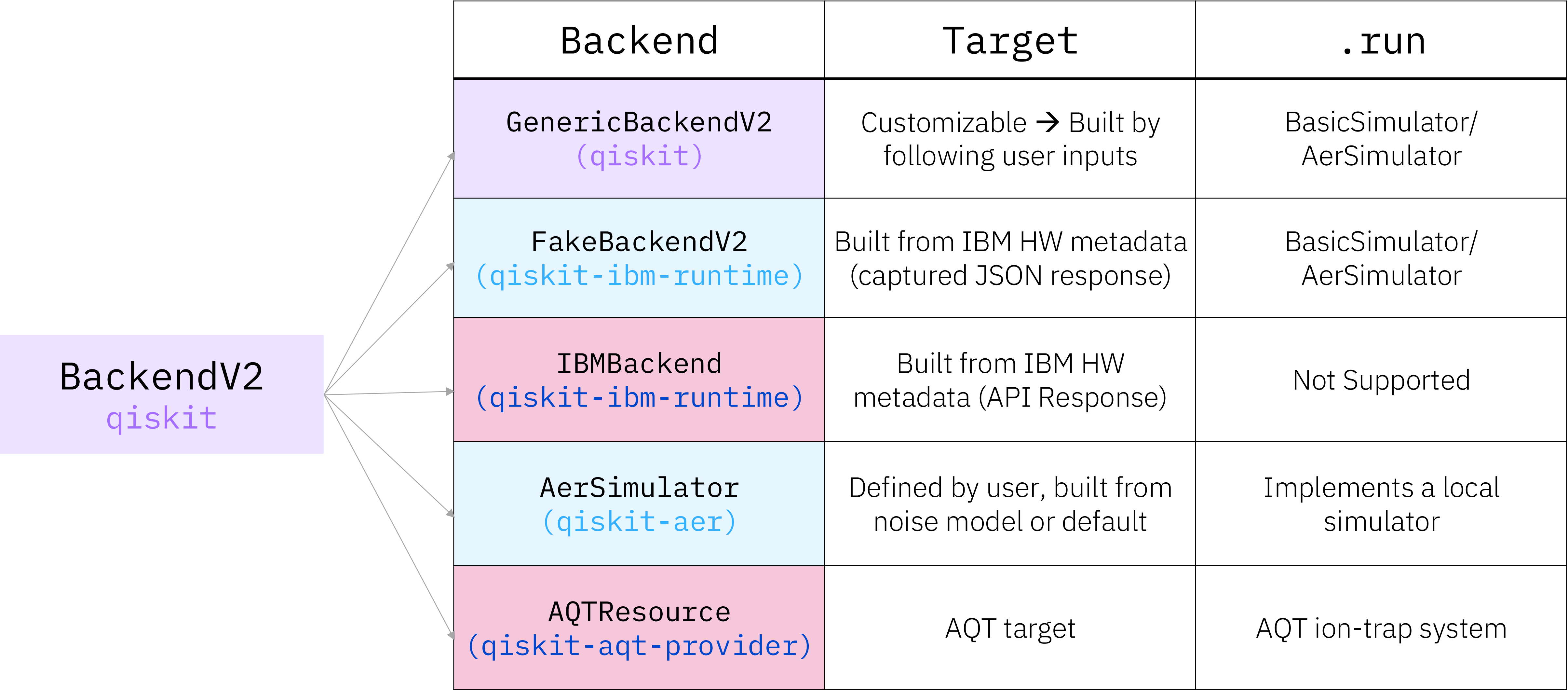
`Backend = Target + .run (+ extra functionality)`

1. Required

2. Optional, 3. Custom

- **Abstract** interface defined in Qiskit SDK
- Contains a `Target` instance → HW description
- Each subclass must implement its own `.run` method → execution
- Vendors can extend the basic interface with `extra functionality`
 - Optional: Qiskit transpiler hooks
 - Custom: For example, configuration and properties attributes in `IBMBackend`

BackendV2 abstract interface implementations



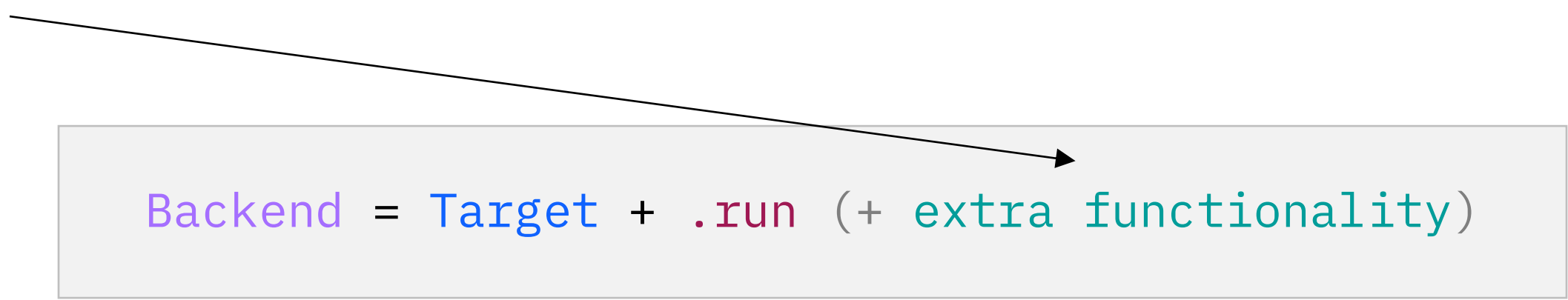
BackendV2 abstract interface implementations

- IBM Runtime for C: <https://github.com/Qiskit/qiskit-ibm-runtime-c>
- IonQ <https://github.com/qiskit-community/qiskit-ionq>
- Cold atom: <https://github.com/qiskit-community/qiskit-cold-atom>
- Amazon Braket: <https://github.com/qiskit-community/qiskit-braket-provider>
- Dell: <https://github.com/qiskit-community/qiskit-dell-runtime>
- Zapata (orquestra): <https://github.com/zapata-engineering/orquestra-qiskit>
- Quantinuum: <https://github.com/qiskit-community/qiskit-quantinuum-provider>
- JKU simulator <https://github.com/qiskit-community/qiskit-jku-provider>
- Quantag: <https://pypi.org/project/quantag/>
- QCGPU simulator: <https://github.com/qiskit-community/qiskit-qcgpu-provider>
- Pasqal: <https://github.com/qiskit-community/qiskit-pasqal-provider>
- Azure: <https://github.com/microsoft/azure-quantum-python>
- Superstaq: <https://github.com/Infleqtion/client-superstaq>
- IQM: <https://github.com/iqm-finland/qiskit-on-iqm>
- CERN Simulator: <https://github.com/CERN-IT-INNOVATION/quantum-gates>
- Qulacs: <https://github.com/Gopal-Dahale/qiskit-qulacs>
- Scaleway: <https://github.com/scaleway/qiskit-scaleway>
- Rigetti: <https://github.com/rigetti/qiskit-rigetti>
- Alibaba: <https://pypi.org/project/acquantum-qiskit/>
- QCTIC: <https://bitbucket.org/fundacionctic/erwin-qiskit>
- qudora: <https://pypi.org/project/qudora-sdk/>
- Alice and Bob: <https://github.com/Alice-Bob-SW/qiskit-alice-bob-provider>
- MQP: <https://pypi.org/project/mqp-qiskit-provider/>
- DDSIM: <https://github.com/munich-quantum-toolkit/ddsim>
- Quantum Inspire: <https://github.com/qutech-delft/qiskit-quantuminspire>
- Kipy quantum hub (Planqk): <https://gitlab.com/planqk-foss/planqk-quantum>
- Quantum Rings: <https://pypi.org/project/QuantumRingsLib/>
- QRyd: <https://pypi.org/project/qiskit-qryd-provider/>
- Qrack: <https://github.com/vm6502q/qiskit-qrack-provider>
- MTBDD: <https://pypi.org/project/qiskit-medusa/>
- OxIonics: <https://pypi.org/project/oxionics-qiskit-provider/>
- SliQSim: <https://github.com/NTU-ALComLab/SliQSim-Qiskit-Interface>
- pQCee: <https://github.com/pqcee/qiskit-pqcee-provider>
- quTree: <https://pypi.org/project/qiskit-qutree-cloud-provider/>
- Symbolic simulator for qiskit circuits: <https://pypi.org/project/symbolic-qiskit/>
- Symbolic simulator for qiskit circuits: <https://pypi.org/project/qiskit-symb/>
- Bosonic simulator: <https://github.com/C2QA/bosonic-qiskit>
- qcentroid: <https://pypi.org/project/qcentroid-runtime-qiskit/>
- strangeworks: <https://pypi.org/project/strangeworks-qiskit/> <https://pypi.org/project/strangeworks-qiskit-runtime/>
- quantastica: <https://pypi.org/project/quantastica-qiskit-toaster/>
- QApp: <https://pypi.org/project/qapp-qiskit/>
- Calcul Québec: <https://pypi.org/project/qiskit-calculquebec/>
- qcware: <https://pypi.org/project/qiskit-qcware/>
- quapp: <https://pypi.org/project/quapp-qiskit/>
- OpenQuantum: <https://pypi.org/project/openquantum-sdk-qiskit/>
- quimb: <https://pypi.org/project/qiskit-quimb/>
- Tergite: <https://pypi.org/project/tergite/>

BackendV2 Custom functionality

- The `IBMQBackend` implementation incorporates **custom attributes** such as:

- `status`
- `configuration`
- `properties`
- `parametric_pulses`
- `credits_required`
- `conditional_latency`
- `processor_type`
- & more



The diagram illustrates the composition of a Backend. A line from the word 'custom' in the first list item points to a box containing the equation: Backend = Target + .run (+ extra functionality). The word 'Backend' is purple, 'Target' is blue, '.run' is red, and 'extra functionality' is teal.

`Backend` = `Target` + `.run` (+ `extra functionality`)

These **extend** the `Target` representation with **vendor-specific** information, and Qiskit does not provide tools to manipulate this information out-of-the-box

Backends and primitives

Backend = Target + "run" (+ extra functionality)

Primitives = "run"

Different abstraction levels

- The run interface in a backend is purposely free-form. It has a bias towards returning **Counts**, which is a first class supported return. But allows any return data from a backend.
 - o Basically, you can think of the Backend run interface as taking a quantum circuit in and returning **something**
- The primitive interface are more prescriptive about inputs and outputs
 - o Has differing levels of abstractions in each primitive but defined I/O types for each primitive
 - o Two current interfaces **Sampler** and **Estimator**
 - o Qiskit provides default implementations for Sampler and Estimator from a **BackendV2** using **Counts**

Target

- The constraints of a given compilation target is modeled by the **Target** class in Qiskit.
- The target contains a list of every instruction (operation/gate on qubits), the properties associated with those instructions. Typically error rate, and instruction duration and any bounds on parameter values.
- Heterogeneous gate sets are supported; define different instructions on differing subsets of qubits
- Additional properties about qubits and other device characteristics use for compilation are contained within the target.

<https://docs.quantum.ibm.com/api/qiskit/qiskit.transpiler.Target>

print(backend.target)

Target			
Number of qubits: 5			
Instructions:			
CX	(0, 1):	Duration: 4.124444444444444e-07 sec. Error Rate: 0.011112942844963669 With pulse schedule calibration	
	(1, 0):	Duration: 3.768888888888884e-07 sec. Error Rate: 0.011112942844963669 With pulse schedule calibration	
	(1, 2):	Duration: 2.702222222222222e-07 sec. Error Rate: 0.007959389035724018 With pulse schedule calibration	
	(2, 1):	Duration: 3.057777777777775e-07 sec. Error Rate: 0.007959389035724018 With pulse schedule calibration	
	(2, 3):	Duration: 2.844444444444443e-07 sec. Error Rate: 0.025418806013513956 With pulse schedule calibration	
	(3, 2):	Duration: 3.2e-07 sec. Error Rate: 0.025418806013513956 With pulse schedule calibration	
	(3, 4):	Duration: 4.195555555555555e-07 sec. Error Rate: 0.018081475298807576 With pulse schedule calibration	
	(4, 3):	Duration: 3.84e-07 sec. Error Rate: 0.018081475298807576 With pulse schedule calibration	
	(0,):	Duration: 3.555555555555554e-08 sec. Error Rate: 0.00037753800551696055 With pulse schedule calibration	

The **Target** class. Overview

- Consumed by **transpiler** → models compilation target constraints (HW specifications)
- Designed to be **vendor-agnostic**, but allows **vendor-specific extensions**
- 3 categories of information:

- | | | | |
|-------------|---|---|-------------------|
| 1. Required | } | Qiskit offers tools to reason about these elements | → vendor-specific |
| 2. Optional | | | |
| 3. Custom | } | Tools are not part of Qiskit, but the model is extensible | |

The **Target** class. HW Information

1. Required:

- `#qubits`
- Supported instructions $\rightarrow$ (`instruction`, `qargs`)

2. Optional:

- `QubitProperties`
- `InstructionProperties` (error rate, duration) associated to (`instruction`, `qargs`)
- Global timing, alignment info

3. Custom: extra properties

Qiskit **offers tools** to reason about these elements

Tools are not part of Qiskit, but the model is extensible

The Target class. HW Information

Required

```
Target
Number of qubits: 5
Instructions:
      cx
      (0, 1):
      (1, 0):
      (1, 2):
      (2, 1):
      (2, 3):
      (3, 2):
      (3, 4):
      (4, 3):
      sx
      (0,):
```

Optional

```
Duration: 4.124444444444444e-07 sec.
Error Rate: 0.011112942844963669

Duration: 3.7688888888888884e-07 sec.
Error Rate: 0.011112942844963669

Duration: 2.702222222222222e-07 sec.
Error Rate: 0.007959389035724018

Duration: 3.0577777777777775e-07 sec.
Error Rate: 0.007959389035724018

Duration: 2.8444444444444443e-07 sec.
Error Rate: 0.025418806013513956

Duration: 3.2e-07 sec.
Error Rate: 0.025418806013513956

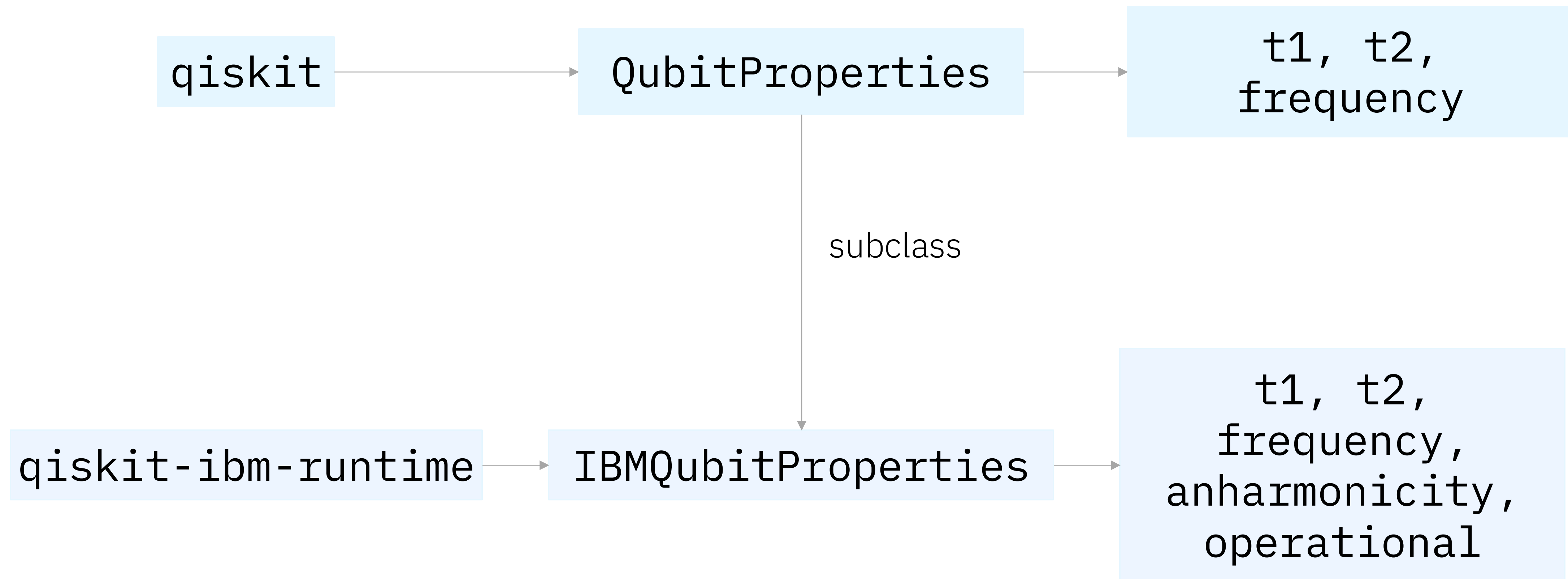
Duration: 4.195555555555555e-07 sec.
Error Rate: 0.018081475298807576

Duration: 3.84e-07 sec.
Error Rate: 0.018081475298807576

Duration: 3.555555555555554e-08 sec.
Error Rate: 0.00037753800551696055
```


The **Target** class. Model Extension

- An example of Target model extension → custom qubit properties



IBM Example

```
from qiskit.circuit.library import QuantumVolume

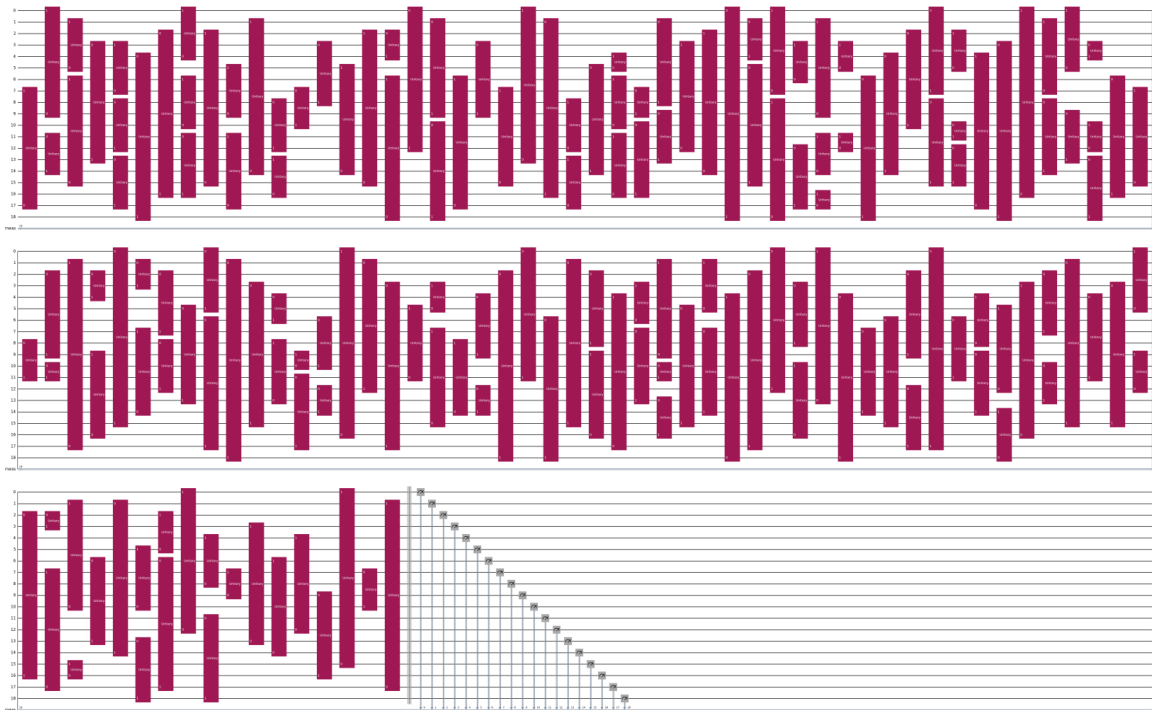
circuit = QuantumVolume(19, seed=42)
circuit.measure_all()
```

```
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit import transpile

service = QiskitRuntimeService(name="performance-premium")
backend = service.backend("ibm_miami")
tqc = transpile(circuit, backend)
```

```
from qiskit_ibm_runtime import Sampler

sampler = Sampler(backend)
sampler.options.default_shots = 4196
job = sampler.run([tqc])
```

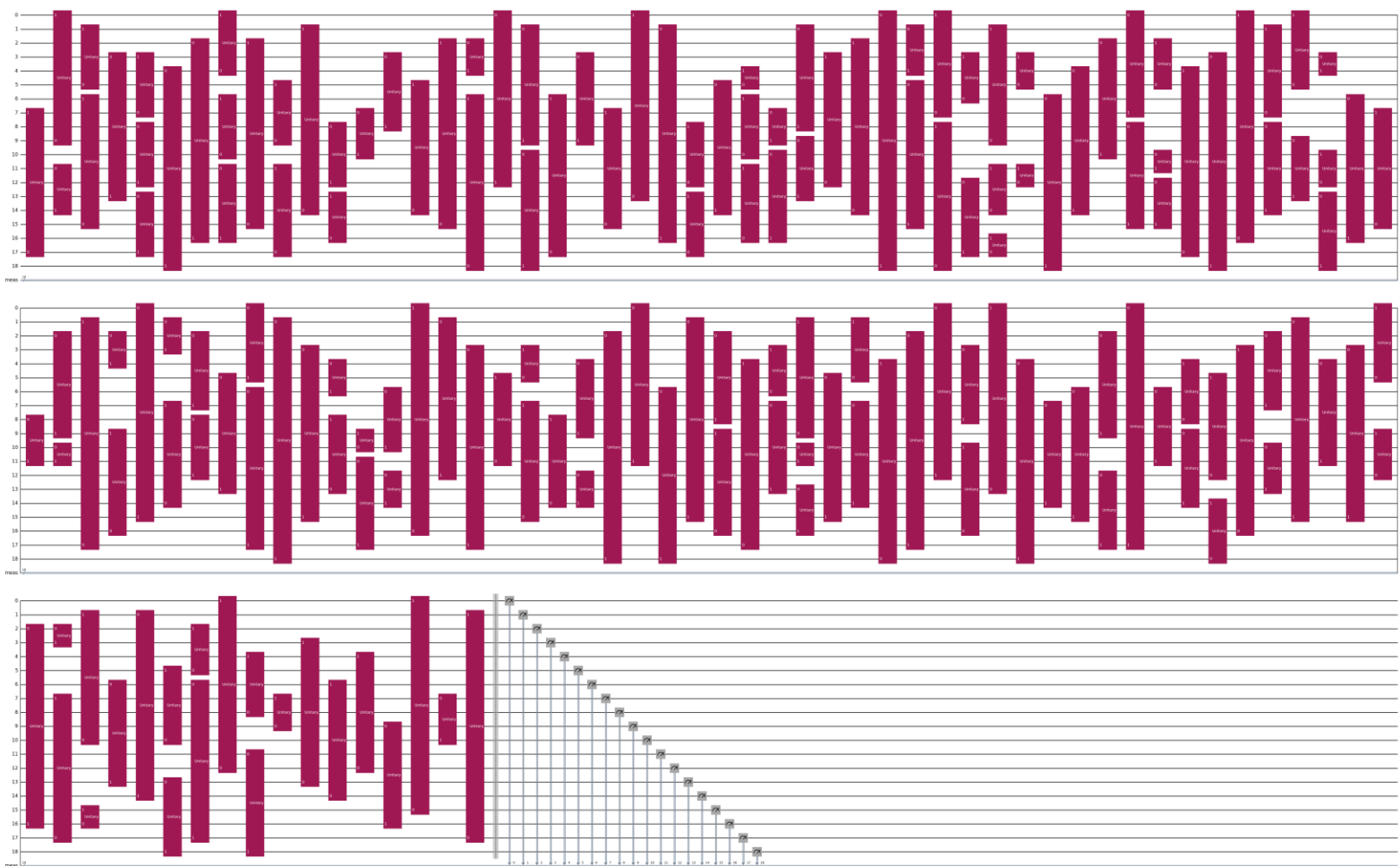


```
{
  "program_id": "sampler",
  "backend": "ibm_pittsburgh",
  "params": {
    "pubs": [
      {
        "__type__": "QuantumCircuit",
        "__value__": "eJzt00dvHlUchfH3dZzYTnF6dZrTe9x77733XnB2IExghdjyedmyAo5znBgUFkdig/T8pa\nvn6s78NLOYmR2dHx9dqCwpFjzF7w8LhQ+F8r+233hpSnTp07Z0tXBifv5lv3h0X2nx24PdJ4W/T/EE15xyS93T7hm3zC\n13K9yz7jn3vHvBrXQvupfcy+4V96p7zb3u3nBvurfc2+4dt8q9695z77sP3IdutfvIfew+cZ+6z9zn7gv3pfvKfe2+cd\n+679z3bo1b69a59W6D2+g2uc1ui9vqtrntbofb6Xa53W6P2+v2uf3ugDvoDrnD7og76o654+6E0+10udPujDvrzrnz7o\nK76C65y+6Ke/yp7nr7oa76W652+60u+vuufuF49EXxyx8+UP+McW5teHdHw80tf/gS/Lvfq367ffqip7Pv8u/4fmVz/\ni/eVDJV6BW/xd8eHQGAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\nAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\nAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA0B/AMp0Xv0KmJzY/fjTD5/E/tFRIIr76TNK9o\n9ePhCnJE4lo1SiNBGnJU4n4ozEmUSUSZQ1oIyIPBEVEhWJ0CtXNhHnJM4l4rzE+URckLiQiEqJyKRclLiYiEsSlxJxWe\nJyIq5IXEnEVYmribgmcS0R1yWuJ+KGxI1E3JS4mYhbErcScVvidiLuSNxJRJVEVSLuStxNxD2Je4m4L3E/EQ8kHiTioc\nTDRFRLVCfikcSjRDyWeJyIJxJPEvFU4mkinkk8S8RzieeJeCHxIhEvJV4m4pXEQ0S8lnidiDcSbxLxVuJtIt5JvEvEe4\nn3iaiRqElErURtIuok6hJRL1GfiAaJhkQ0SjQmokmiKRHNes2JaJFoSUSrRGsi2iTaEtEu0Z6IDomORHRKdCaiS6IrEd\n0S3YnokehJRK9EbyL6JPS0S/Rn4gBiYFEDEoMJmJIYigRwxLDiRiRGEnEqMRoIsYkxhIxLjGeiAmJiURMSkwmYkpiKh\nHTEt0JmJGYScSsxGwi5iTmEjEvMZ+IBYmFRCxKLCZiSWIpEcsSy4lYkVhJxKrEaiLWJNYSS6xnogNiY1EbEpsJmJLYi\nsR2xLbidiR2EnErsRuIvYk9hKxL6H1x4k5vvFPkmfQPA=="
      }
    ]
  },
  "shots": 4196,
  "version": 2
},
{"version": 2}
```


AQT example

```
from qiskit.circuit.library import QuantumVolume

circuit = QuantumVolume(19, seed=42)
circuit.measure_all()
```

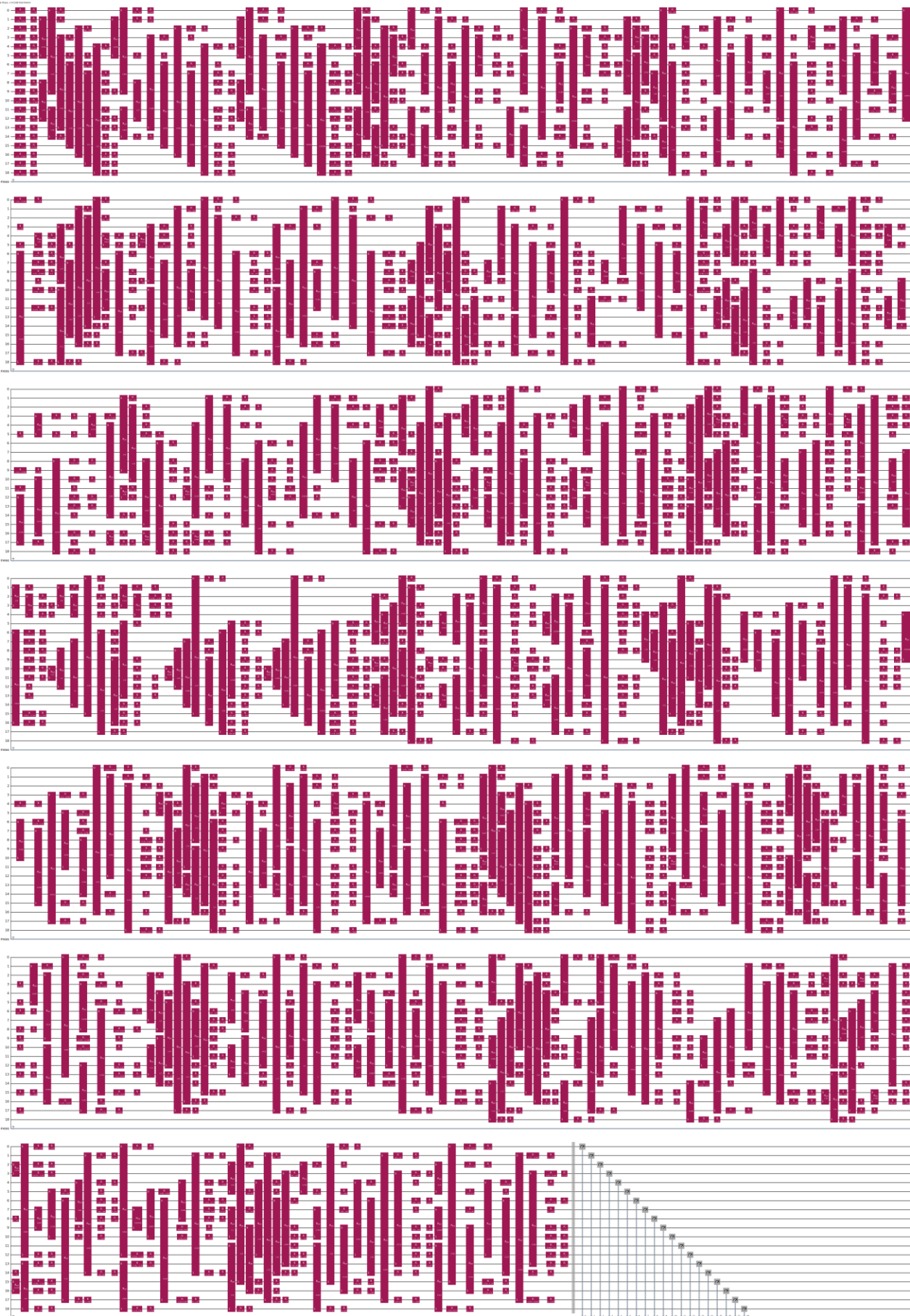


```
target = Target(num_qubits=20)
theta = Parameter("θ")
phi = Parameter("φ")
lam = Parameter("λ")

target.add_instruction(RZGate(lam))
target.add_instruction(RGate(theta, phi))
target.add_instruction(
    RXXGate(theta,
    angle_bounds=[(0, pi / 2)]
)
target.add_instruction(Measure())
```

```
from qiskit_aqt_provider import AQTProvider
from qiskit import transpile

provider = AQTProvider("TOKEN")
backend = provider.get_backend("marmot")
tqc = transpile(circuit, backend)
```



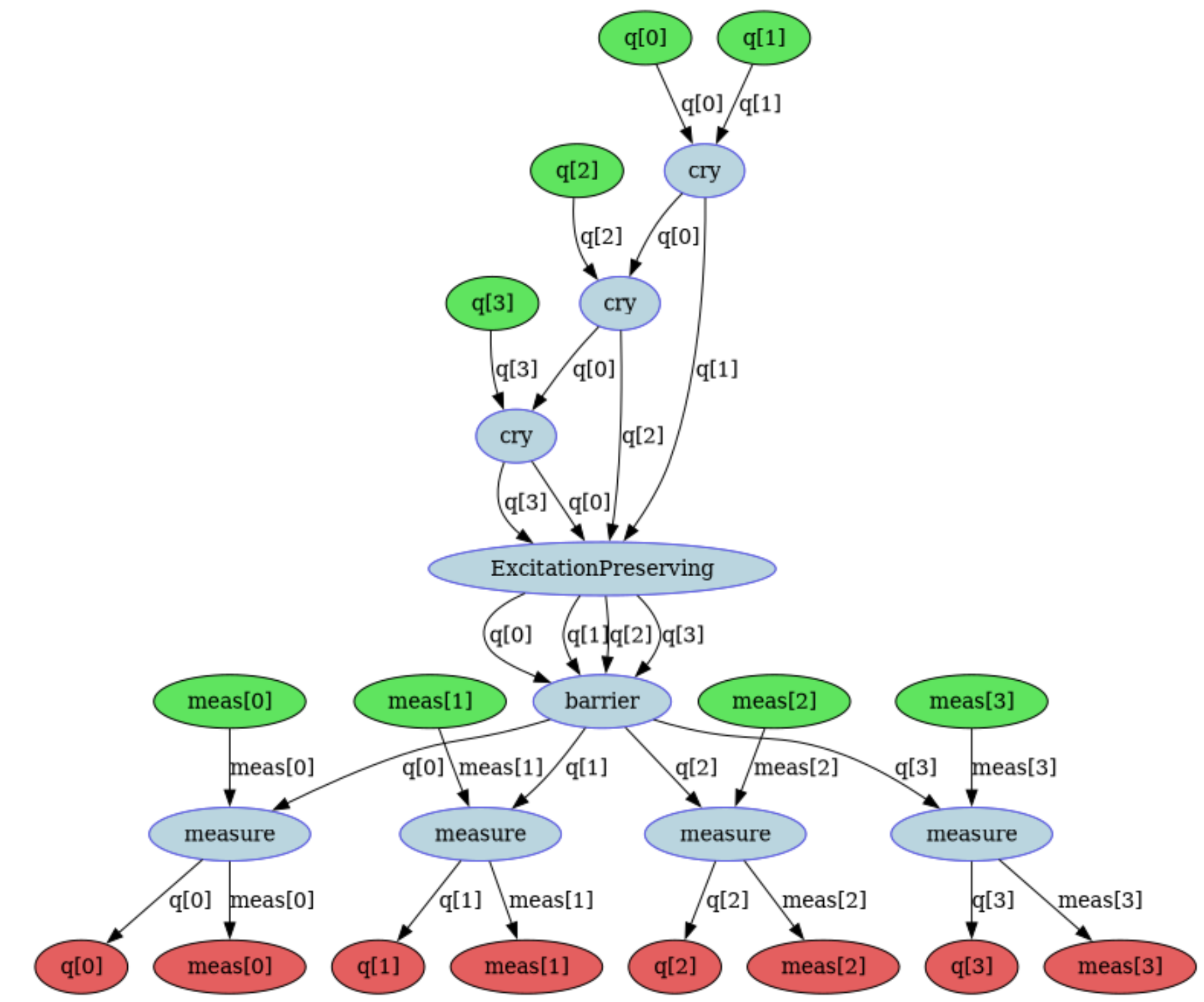
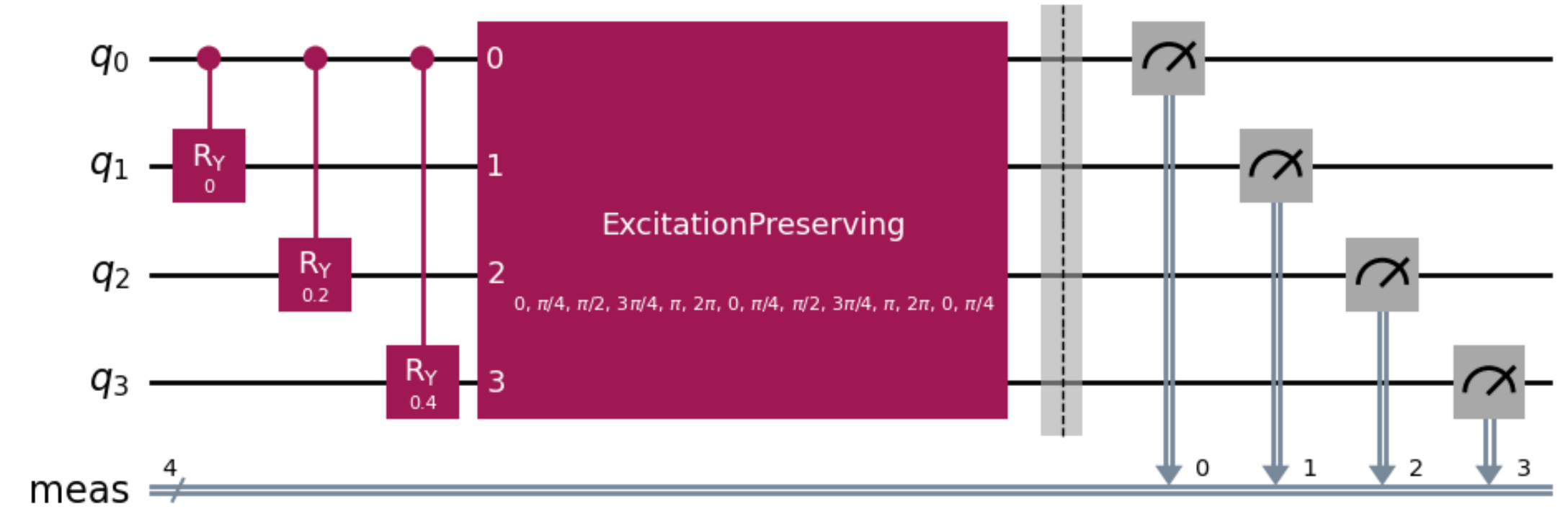
```
backend.run(tqc)
```

```
[
  {
    "operation": "R",
    "phi": 0.6512386422782583,
    "theta": -1.8041100267730634,
    "qubit": 0
  },
  {
    "operation": "R",
    "phi": 1.9096793613603733,
    "theta": 3.141592653589793,
    "qubit": 0
  },
  {
    "operation": "R",
    "phi": 0.3913736390588314,
    "theta": -1.3591217728163585,
    "qubit": 1
  },
  {
    "operation": "R",
    "phi": 1.7335661509619662,
    "theta": 3.141592653589793,
    "qubit": 1
  },
]
```

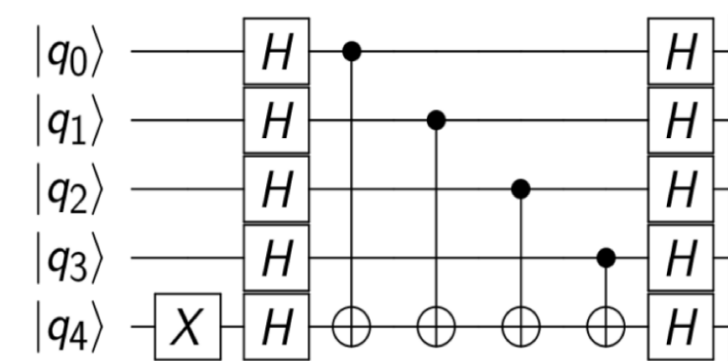
Qiskit Transpiler

DAG Representation

- Qiskit represents circuits internally as Directed Acyclic Graphs (**DAGs**) for compilation
- The **DAGCircuit** class tracks the **flow of data** from **input nodes** through **operations** to **output nodes**
- The **edges** track bits (quantum/classical) and other classical variables
- This representation simplifies **circuit analysis**, allows to **see dependencies between operations** and simplifies in-place **mutation and substitution** of operations.
- This is the default IR used inside Qiskit's compiler but the infrastructure for compilation is IR independent.



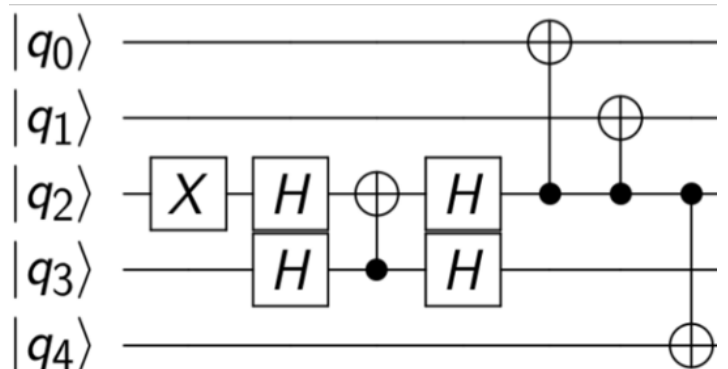
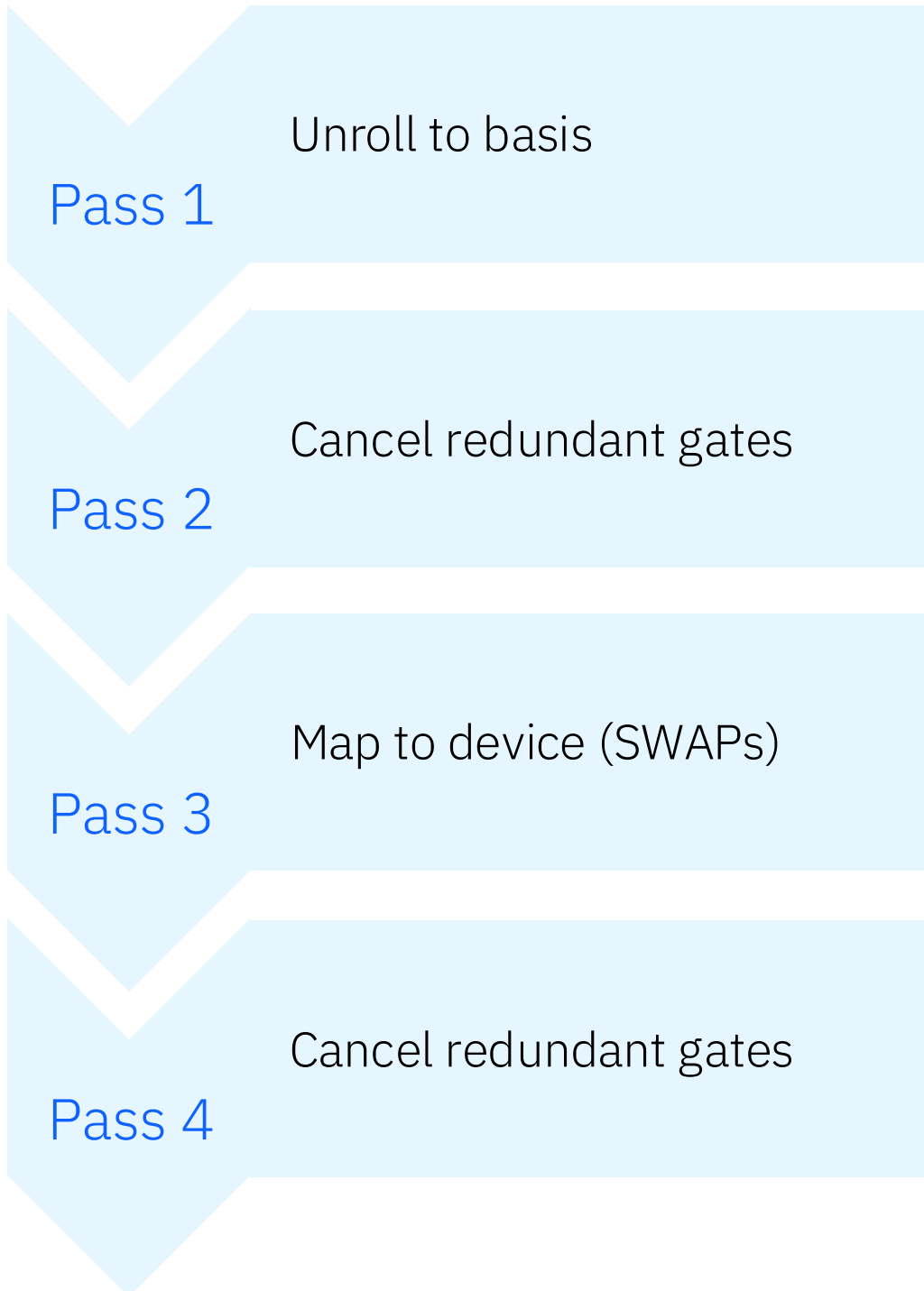
Transpiler Architecture



The transpiler pipeline is built from a series of *Passes*.

Each pass does a small, well-defined task.

Qiskit has a library of passes, but there is also an abstract interface to write custom passes.

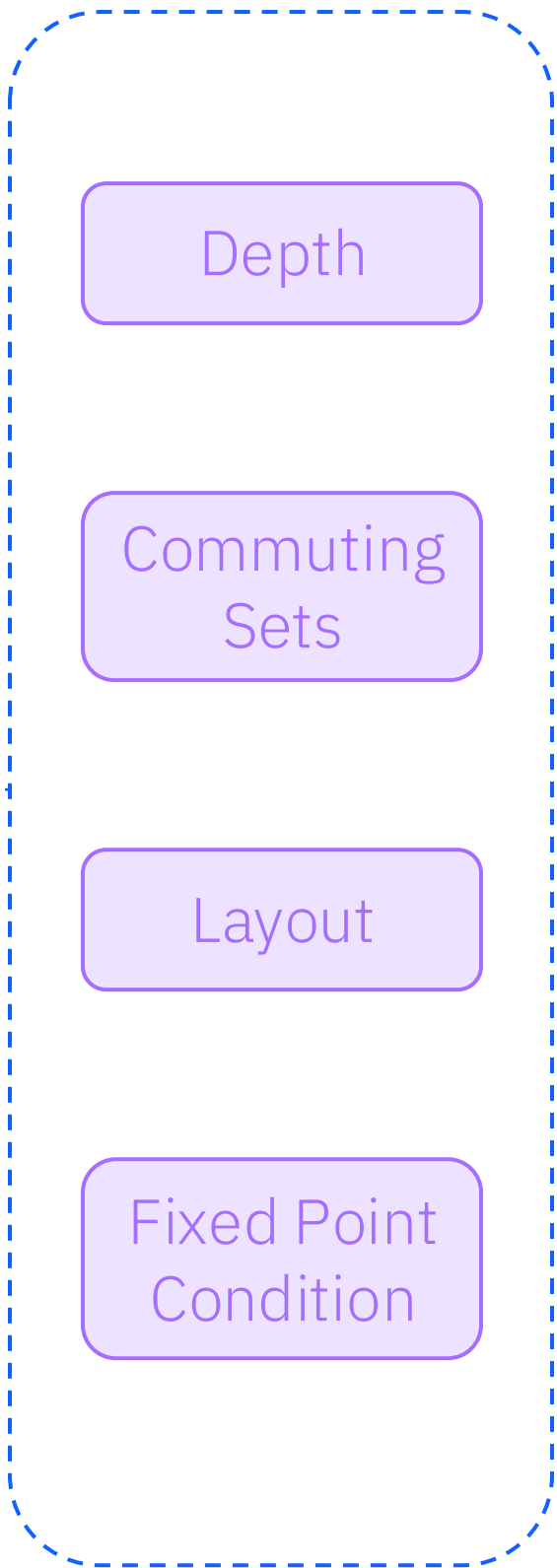


The `PassManager` abstraction deals with the complexity of scheduling the execution of passes. It supports control flow around the execution of passes.



Qiskit exposes a series of *Preset Pass Managers* that are already configured.

Global Property Set



The *Pass Manager* maintains a global context about the circuit (*Property Set*) as it runs through the pipeline.

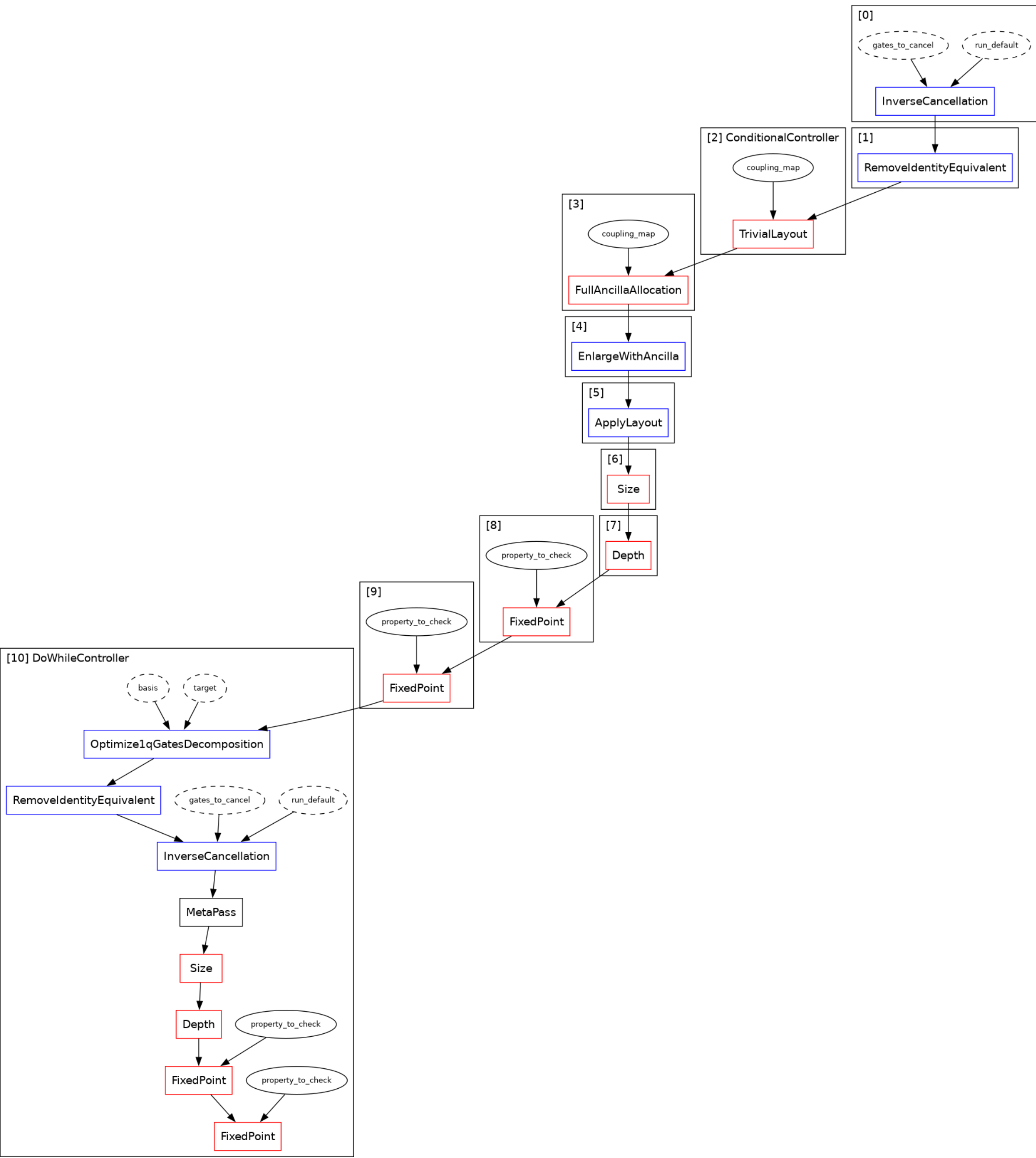

```
from qiskit.transpiler import passes, Target, PassManager
from qiskit.passmanager.flow_controllers import ConditionalController, DoWhileController

fake_target = Target.from_configuration(num_qubits=1000, basis_gates=["cx", "u"])

def choose_layout_condition(property_set):
    return property_set["layout"] is None

def check_fixed_point():
    return not (property_set["depth_fixed_point"] and property_set["size_fixed_point"])

def build_passmanager(target: Target):
    fixed_point_size = [
        passes.Size(recurse=True),
        passes.Depth(recurse=True),
        passes.FixedPoint("size"),
        passes.FixedPoint("depth"),
    ]
    pm = PassManager([passes.InverseCancellation(), passes.RemoveIdentityEquivalent()])
    pm.append(
        ConditionalController(
            passes.TrivialLayout(target), condition=choose_layout_condition
        )
    )
    pm.append(
        [
            passes.FullAncillaAllocation(target),
            passes.EnlargeWithAncilla(),
            passes.ApplyLayout(),
        ]
    )
    pm.append(fixed_point_size)
    pm.append(
        DoWhileController(
            [
                passes.Optimize1qGatesDecomposition(target=target),
                passes.RemoveIdentityEquivalent(),
                passes.InverseCancellation(),
                passes.MetaPass(),
                passes.Size(),
                passes.Depth(),
                passes.FixedPoint(),
            ]
            + fixed_point_size,
            do_while=check_fixed_point,
        )
    )
    return pm
```



Transpiler Modularity

- The default Qiskit compilation pipeline is divided into discrete **stages** via the **StagedPassManager** class
- The stages execute linearly, and each performs a logical transformation
- The initial stage takes an **abstract circuit**
- After the final stage, the output is optimized for execution on a given target
- Qiskit's **Preset Pass Managers** use **6 stages**
- If you're constructing a pass manager manually, you can define your own stages

Docs: <https://docs.quantum.ibm.com/guides/transpiler-stages>

1. `init` stage

- Logical optimizations
- Decomposing larger gates into 1q and 2q gates

2. `layout` stage

- Mapping circuit qubits into qubits in target

3. `routing` stage

- Solves connectivity constraints.
- Inserts SWAP gates/ other ops. to move state between qubits

4. `translation` stage

- Transforms circuit operations into operations supported by target

5. `optimization` stage

- Optimizes number of gates in the circuit
- Earlier stages likely inserted extra operations

6. `scheduling` stage

- Final stage
- Account for timing of operations in the circuit

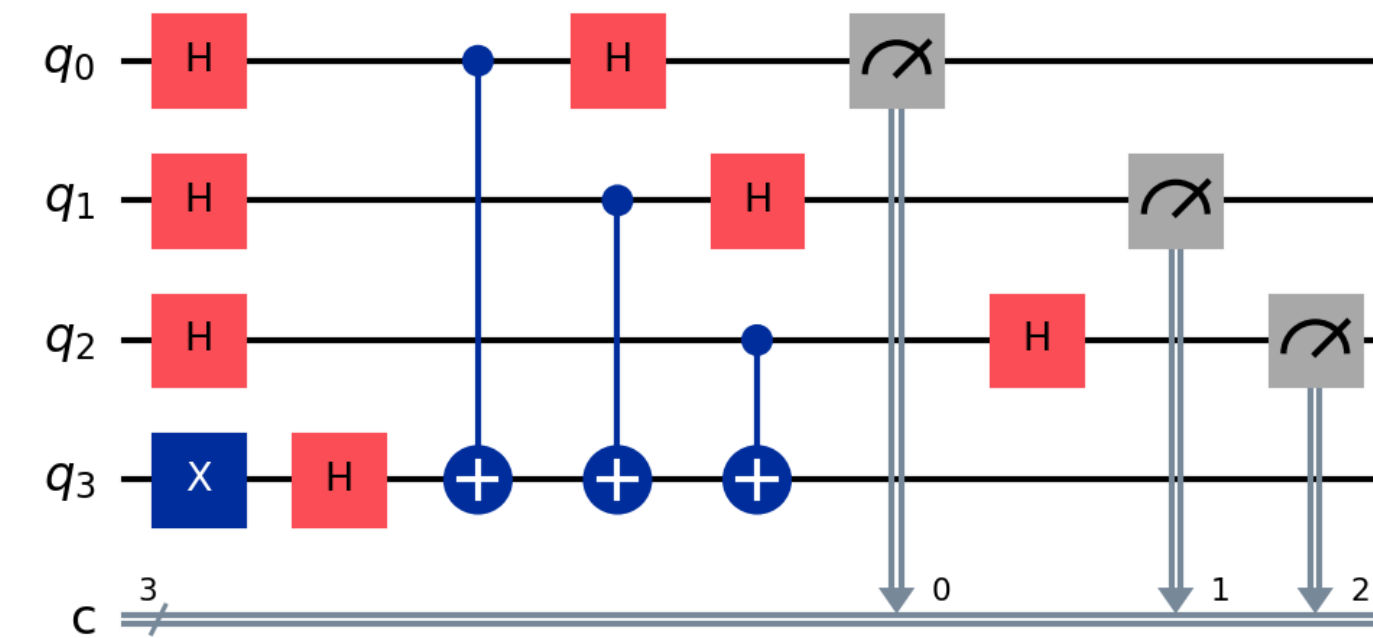
Transpiler Pluggability

- The Pass Managers are **pluggable** and **extensible**
- Users can replace the implementation of a single stage on the fly by:
 - Substituting stage with custom **PassManager** object
 - Modifying stage in-place
- This extensibility allows users to **reuse a prebuilt pipeline and customize a small component**
- Plugins can be installed/published as external packages and included in the Qiskit ecosystem

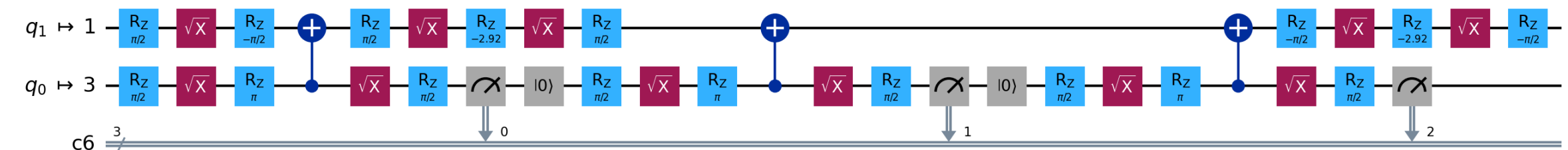
Docs: <https://docs.quantum.ibm.com/guides/transpiler-plugins>

Example: Qubit Reuse Plugin

```
pip install qiskit-qubit-reuse
```



```
pm = generate_preset_pass_manager(2, backend, init_method="qubit_reuse")  
pm.run(qc)
```



The BackendV2 transpiler plugin defaults

- The Qiskit transpiler accepts hooks for custom **translation** and **scheduling** methods
- These hooks **replace** the **default plugins** used by Qiskit
- Useful for backends with constraints that are **not expressible via Target**
- This is an example of **optional Backend** functionality



```
Backend = Target + .run (+ extra functionality)
```

- To “unlock” this capability for a backend, add methods:
 - `get_translation_stage_plugin()`
 - `get_scheduling_stage_plugin()`

The BackendV2 transpiler plugin defaults

- Example of translation hook with custom plugins:

```
class IBMBackend(...):  
  
    ...  
  
    def get_translation_stage_plugin(self) -> str:  
        """Return the default translation stage plugin name for IBM backends."""  
        if not self.options.use_fractional_gates:  
            return "ibm_dynamic_circuits"  
        return "ibm_fractional"
```

Extending the Transpiler for custom gates

- In addition to the pluggability it is possible to extend the transpiler to handle custom operations without plugins.
- In the preset pass managers a single pass `BasisTranslator` is responsible for translating gates into those that comply with the target.
- There is a default library of equivalence relationships used by the pass which can be extended with custom gates
- This will enable the transpiler to target arbitrary gates and produce functional circuits.
- Doesn't enable optimizations using the gates.

Docs: <https://quantum.cloud.ibm.com/docs/en/api/qiskit/providers#custom-basis-gates>

```
import numpy as np

from qiskit.circuit.singleton import SingletonGate
from qiskit.circuit import QuantumCircuit
from qiskit.circuit.equivalence_library import SessionEquivalenceLibrary
from qiskit.circuit.library import HGate, ZGate, RZGate, U2Gate

FRAC_1_SQRT_2 = 0.707106781186547524400844362104849039
RY_ARRAY = [[FRAC_1_SQRT_2, -FRAC_1_SQRT_2], [FRAC_1_SQRT_2, FRAC_1_SQRT_2]]

class SYGate(SingletonGate):
    def __init__(self, label=None):
        super().__init__("sy", 1, [], label=label)

    def _define(self):
        qc = QuantumCircuit(1)
        qc.ry(pi / 2, 0)
        self.definition = qc

    def __array__(self, dtype=None, copy=None):
        if dtype is None:
            dtype = np.complex128
        return np.array(RY_ARRAY, dtype=dtype, copy=copy)

# H => Z SY
q = qiskit.QuantumRegister(1, "q")
def_sy_h = qiskit.QuantumCircuit(q)
def_sy_h.append(ZGate(), [q[0]], [])
def_sy_h.append(SYGate(), [q[0]], [])
SessionEquivalenceLibrary.add_equivalence(
    HGate(), def_sy_h)

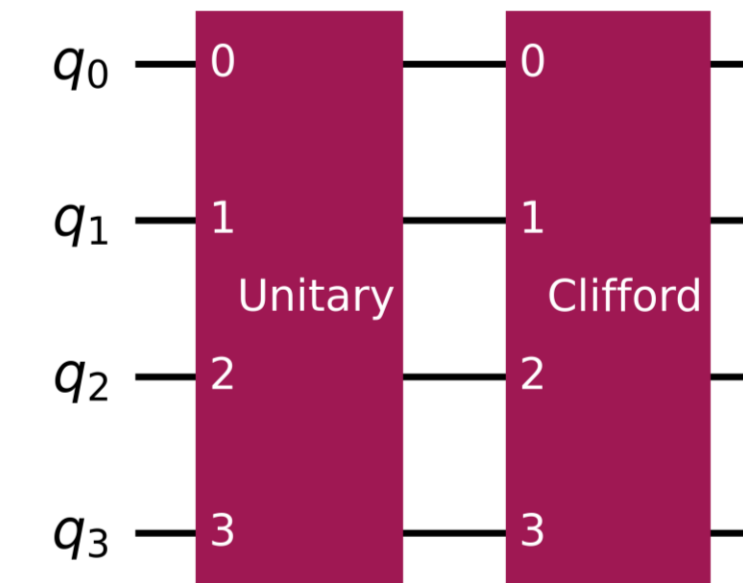
# u2 => Z SY Z
phi = qiskit.circuit.Parameter('phi')
lam = qiskit.circuit.Parameter('lambda')
q = qiskit.QuantumRegister(1, "q")
def_sy_u2 = qiskit.QuantumCircuit(q)
def_sy_u2.append(RZGate(lam), [q[0]], [])
def_sy_u2.append(SYGate(), [q[0]], [])
def_sy_u2.append(RZGate(phi), [q[0]], [])
SessionEquivalenceLibrary.add_equivalence(
    U2Gate(phi, lam), def_sy_u2)
```


Synthesis Plugins

- Qiskit provides two plugin interfaces for synthesizing abstract operations in a circuit: Unitary Synthesis Plugins and High Level Synthesis
- Unitary synthesis plugins are designed to take the matrix from **UnitaryGate** objects (a gate represented solely by it's unitary matrix) and return a circuit implementing that unitary.
- Unitary synthesis plugins are used by the **UnitarySynthesis** transpiler pass.
- High level synthesis plugins are plugins that take any operation object as an input and return a synthesized circuit implementing that object.
- HLS is designed to be open ended to enable custom synthesis of any abstract operation.
- HLS plugins are used by the **HighLevelSynthesis** transpiler pass.

```
from qiskit.circuit import QuantumCircuit
from qiskit.quantum_info.random import random_unitary, random_clifford

qc = QuantumCircuit(4)
qc.unitary(random_unitary(16), qc.qubits)
qc.append(random_clifford(4), qc.qubits)
```

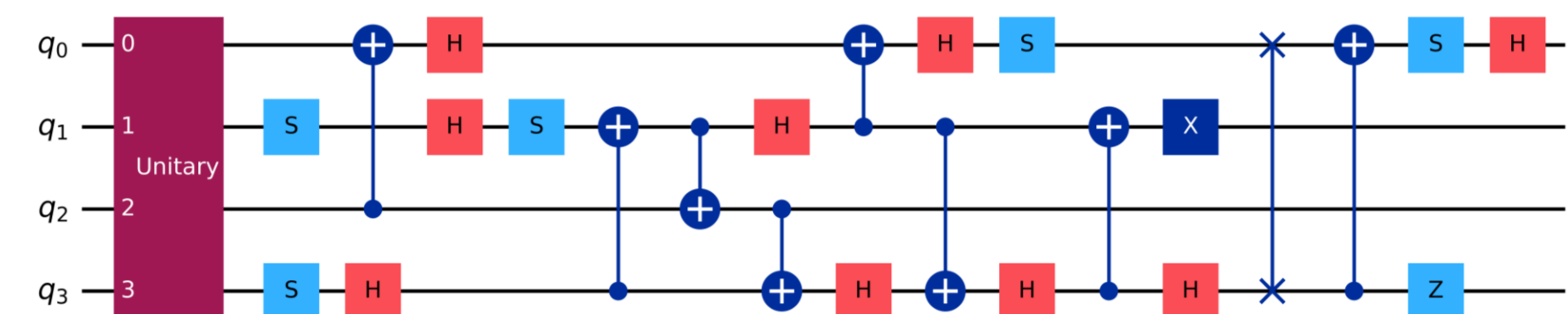


pip install qiskit-sat-synthesis

```
from qiskit.transpiler.passes import UnitarySynthesis, HighLevelSynthesis, HLSConfig

config = HLSConfig(linear_function=[("sat_depth", {"verbosity": 1, "optimize_2q_gates": True})])
hls_pass = HighLevelSynthesis(hls_config=config)
unitary_pass = UnitarySynthesis(method="aqc")

clifford_qc = hls_pass(qc)
```



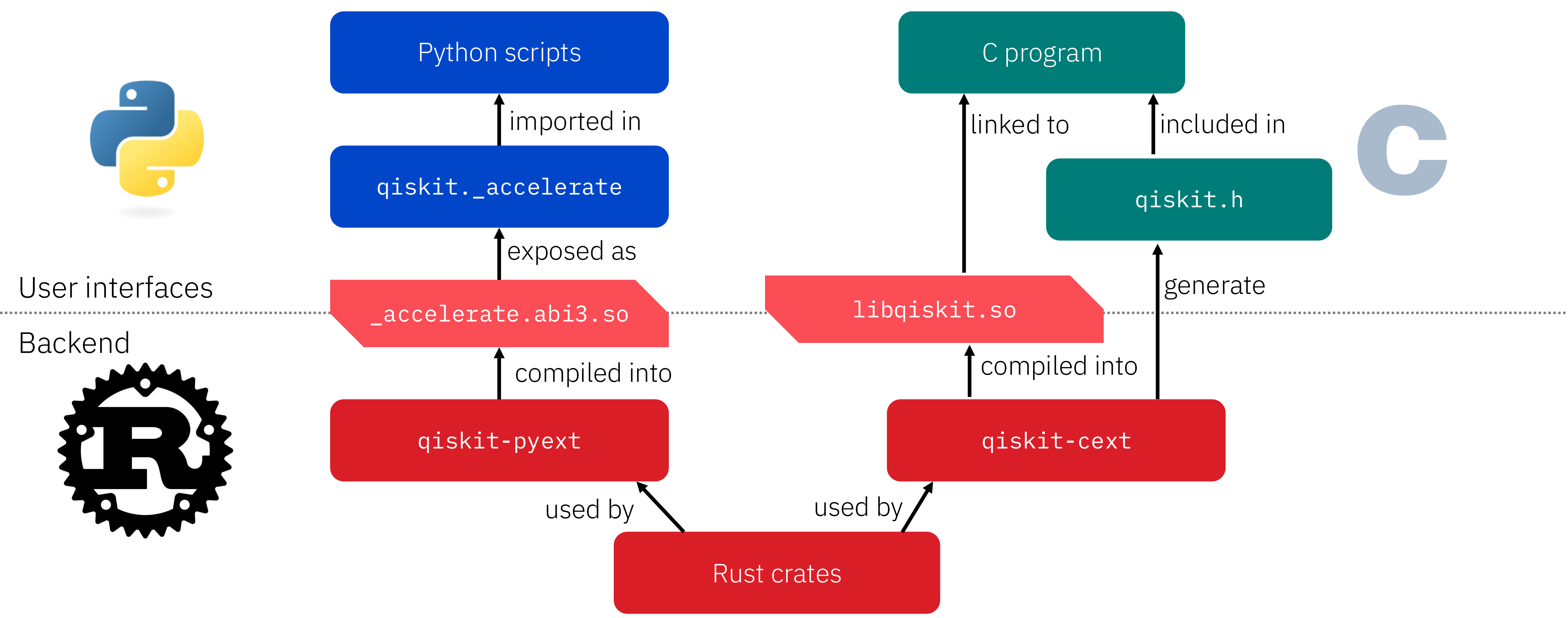
synth_qc = unitary_pass(qc)

Docs:

- https://quantum.cloud.ibm.com/docs/en/api/qiskit/transpiler_synthesis_plugins

Qiskit C API

Qiskit User interfaces



Native Language bindings to Qiskit C API

Qiskit provides interfaces for Python and C

```
from qiskit import QuantumCircuit
```

```
def main():
```

```
    # Build a Bell state
```

```
    qc = QuantumCircuit(2, 2)
```

```
    qc.h(0)
```

```
    qc.cx(0, 1)
```

```
    qc.measure(0, 0)
```

```
    qc.measure(1, 1)
```



```
#include <qiskit.h>
```

```
int main(int argc, char *argv[]) {
```

```
    // Build a Bell state
```

```
    QkCircuit *qc = qk_circuit_new(2, 2);
```

```
    uint32_t h_qargs[1] = {0};
```

```
    qk_circuit_gate(qc, QkGate_H, h_qargs, NULL);
```

```
    uint32_t cx_qargs[2] = {0, 1};
```

```
    qk_circuit_gate(qc, QkGate_CX, cx_qargs, NULL);
```

```
    for (uint32_t i = 0; i < 2; i++) {
```

```
        qk_circuit_measure(qc, i, i);
```

```
    }
```

```
    qk_circuit_free(qc);
```

```
    return 0;
```

```
}
```



Native Language bindings to Qiskit C API

- C++
 - <https://github.com/Qiskit/qiskit-cpp>

```
#include "circuit/quantumcircuit.hpp"
```

```
using namespace Qiskit::circuit;
```

```
int main() {  
    QuantumRegister qr(2);  
    ClassicalRegister cr(2);  
    QuantumCircuit circ(qr, cr);  
    circ.h(0);  
    circ.cx(0, 1);  
    circ.measure(0, 0);  
    circ.measure(1, 1);
```

```
    return 0;
```

```
}
```



Native Language bindings to Qiskit C API

- C++
 - <https://github.com/Qiskit/qiskit-cpp>
- Rust
 - <https://github.com/Qiskit/qiskit-rs>

```
use qiskit_rs::QuantumCircuit;

pub fn main() {
    let mut qc = QuantumCircuit::new(2, 2);
    qc.h(0);
    qc.cx(0, 1);
    qc.measure(0, 0);
    qc.measure(1, 1);
}
```



Native Language bindings to Qiskit C API

- C++
 - <https://github.com/Qiskit/qiskit-cpp>
- Rust
 - <https://github.com/Qiskit/qiskit-rs>
- Julia
 - <https://github.com/Qiskit/Qiskit.jl/>

using Qiskit

```
function build_bell()
  qc = Qiskit.Circuit(2, 2)
  qc.h(1)
  qc.cx(1, 2)
  qc.measure(1, 1)
  qc.measure(2, 2)
  qc
end
```



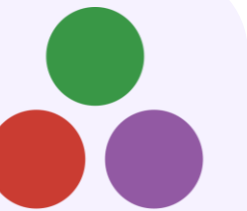
Native Language bindings to Qiskit C API

- C++
 - <https://github.com/Qiskit/qiskit-cpp>
- Rust
 - <https://github.com/Qiskit/qiskit-rs>
- Julia
 - <https://github.com/Qiskit/Qiskit.jl/>

Insert Your Language of choice here

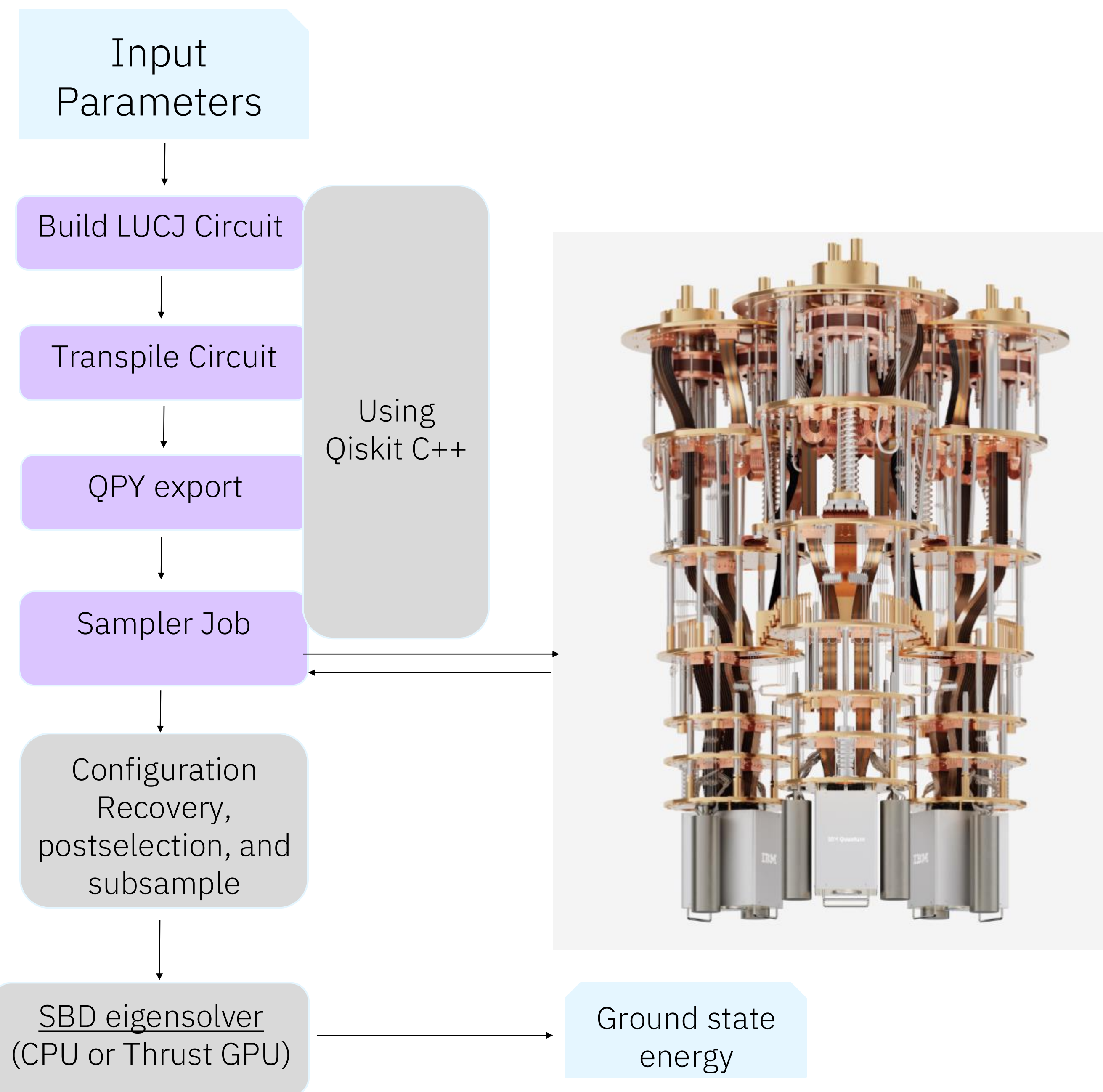
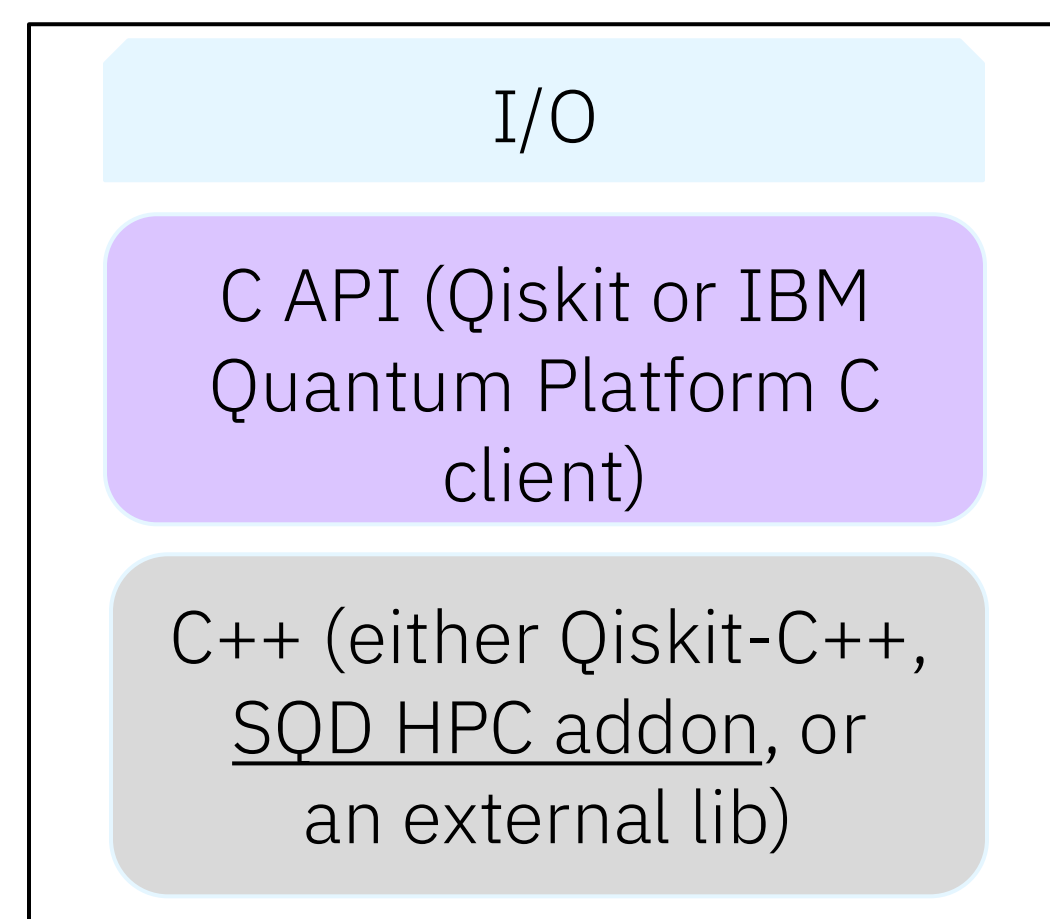
using Qiskit

```
function build_bell()  
  qc = Qiskit.Circuit(2, 2)  
  qc.h(1)  
  qc.cx(1, 2)  
  qc.measure(1, 1)  
  qc.measure(2, 2)  
  qc  
end
```



Qiskit with HPC demo

- Demonstrate SQD workflow described in: Javier Robledo-Moreno *et al.*, Chemistry beyond the scale of exact diagonalization on a quantum-centric supercomputer. *Sci. Adv.* **11**, eadu9991(2025). DOI: [10.1126/sciadv.adu9991](https://doi.org/10.1126/sciadv.adu9991)
- A single binary application that runs classical side as an MPI workload on a cluster and submits jobs to quantum system
- Leverage Qiskit C++ interface to use one language for entire Quantum + HPC workflow
- Demo code can be found here: <https://github.com/qiskit-community/qiskit-c-api-demo>



Questions?

- Github Repository: <https://github.com/Qiskit/qiskit>
- Project page: <https://www.ibm.com/quantum/qiskit>
- Documentation Guides: <https://docs.quantum.ibm.com/guides>
- API Documentation: <https://docs.quantum.ibm.com/api/qiskit>



<https://arxiv.org/abs/2405.08810>